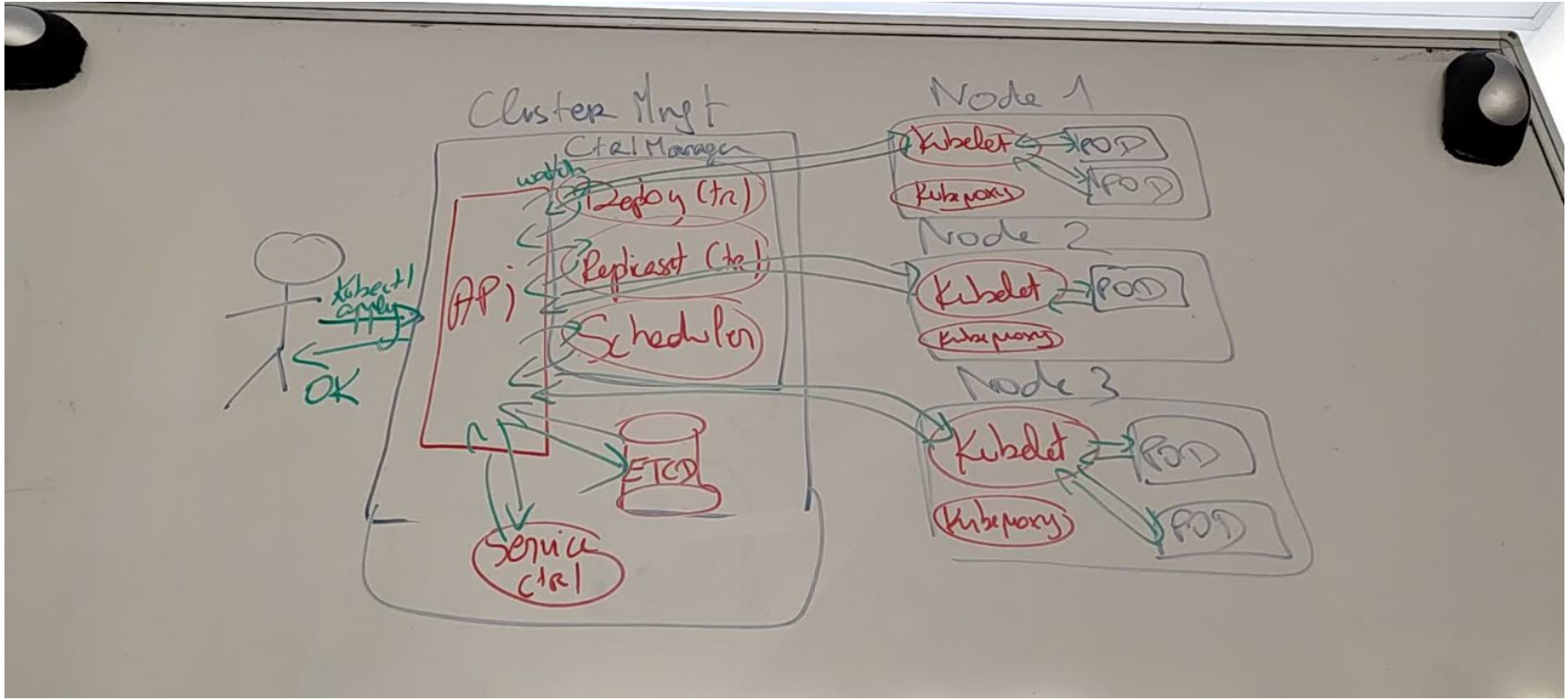
Anatomie de Kubernetes : ce qu'on aurait aimé comprendre plus tôt

Léa, OPS

Simon, aussi OPS

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

SwiftOnSecurity
@SwiftOnSecurity

One time I tried to explain Kubernetes to someone.
Then we both didn't understand it.

16:40 · 06/08/2019 · Twitter for iPhone

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

Mémoriser, pratiquer, comprendre

Les Faits	Les Procédures	Les Concepts
Ce qu'il faut apprendre par coeur (sans forcément avoir besoin de le comprendre)	Ce qu'il faut pratiquer	Ce qu'il faut comprendre en profondeur
Ex : Au feu rouge on s'arrête, la vitesse est limitée à 50 en ville	*Ex : démarrer en côte, faire un créneau*	*Ex: Comment un moteur transforme l'essence en mouvement*

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

* Issue du livre Ultralearning de Scott H. Young

Mémoriser, pratiquer, comprendre

Les Faits

Ce qu'il faut apprendre par coeur

(sans forcément avoir besoin de le comprendre)

Ex : Au feu rouge on s'arrête, la vitesse est limitée à 50 en ville

Les Procédures

Ce qu'il faut pratiquer

Ex : démarrer en côte, faire un créneau

Les Concepts

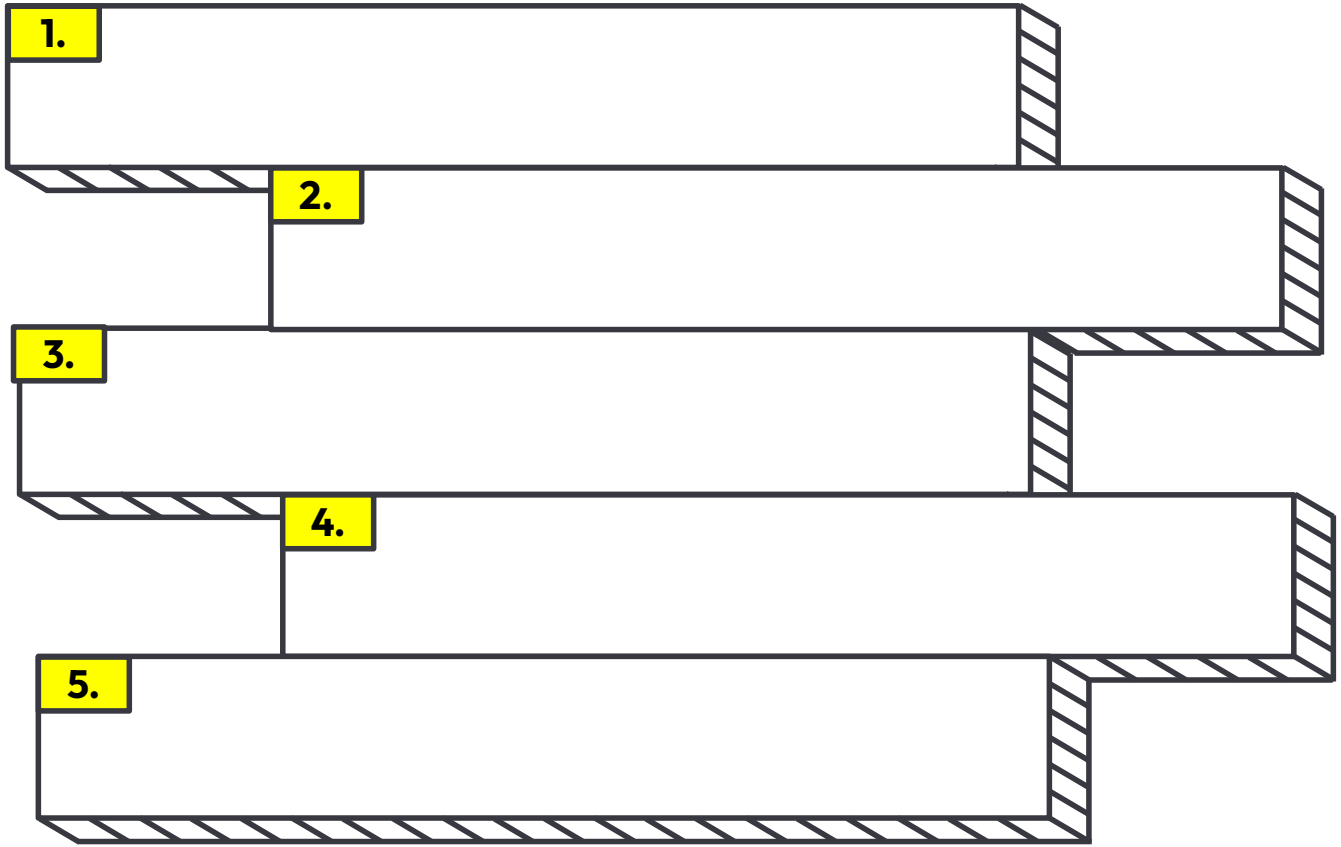
Ce qu'il faut comprendre en profondeur

Ex: Comment un moteur transforme l'essence en mouvement

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

* Issue du livre Ultralearning de Scott H. Young

5 principes d'architecture de Kubernetes



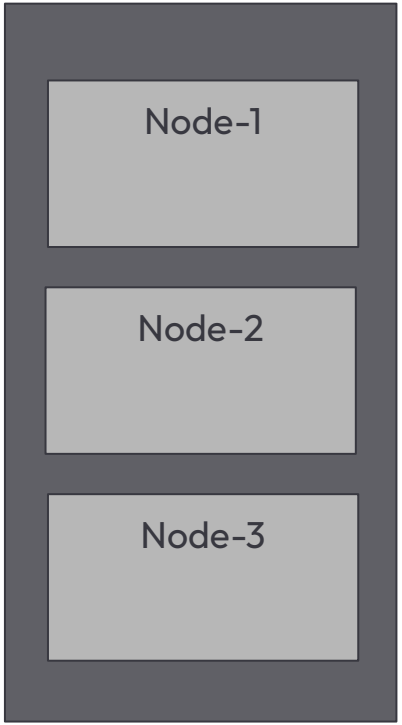
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

App Manager : version 0

La version la plus simple possible

App Manager : version 0

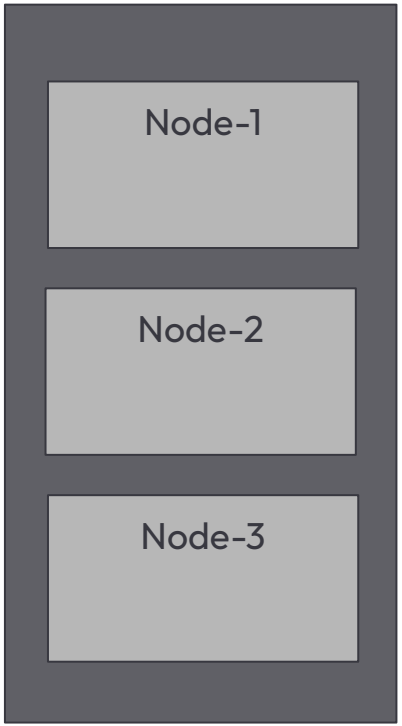
La version la plus simple possible



App Manager : version 0

La version la plus simple possible

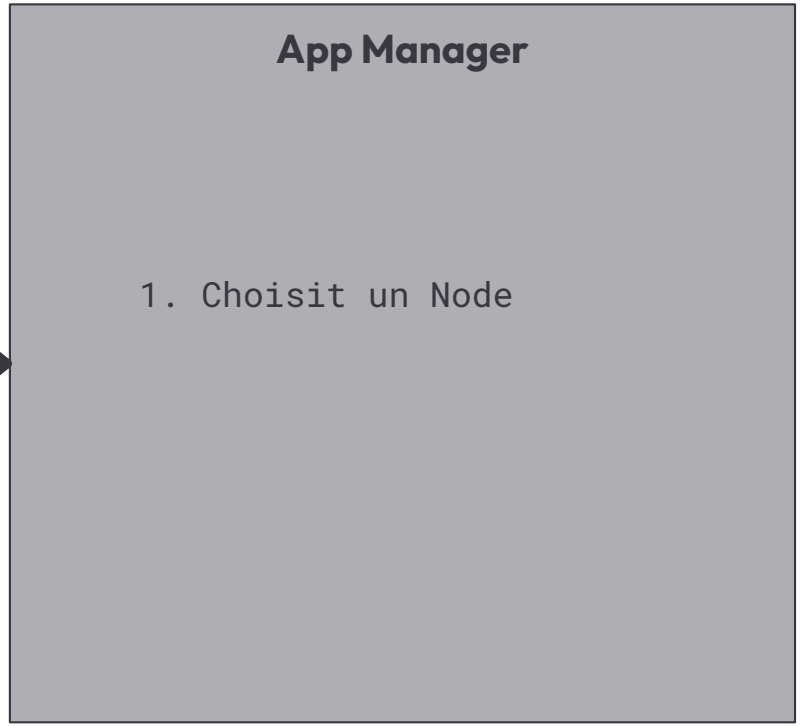
```
PUT /app/<name>
{,
"image": "nginx"
}
```

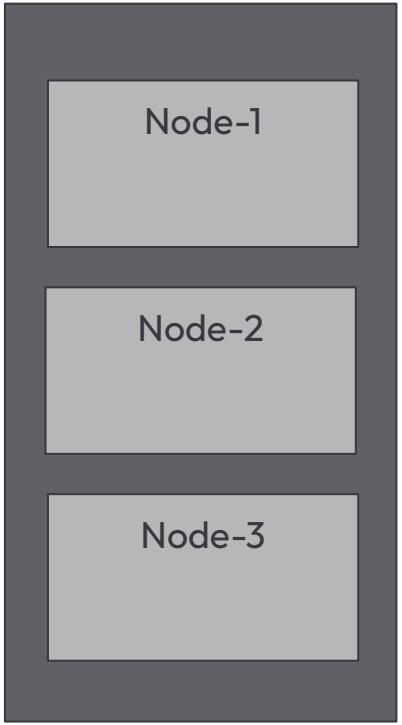


App Manager : version 0

La version la plus simple possible

```
PUT /app/<name>
{,
"image": "nginx"
}
```

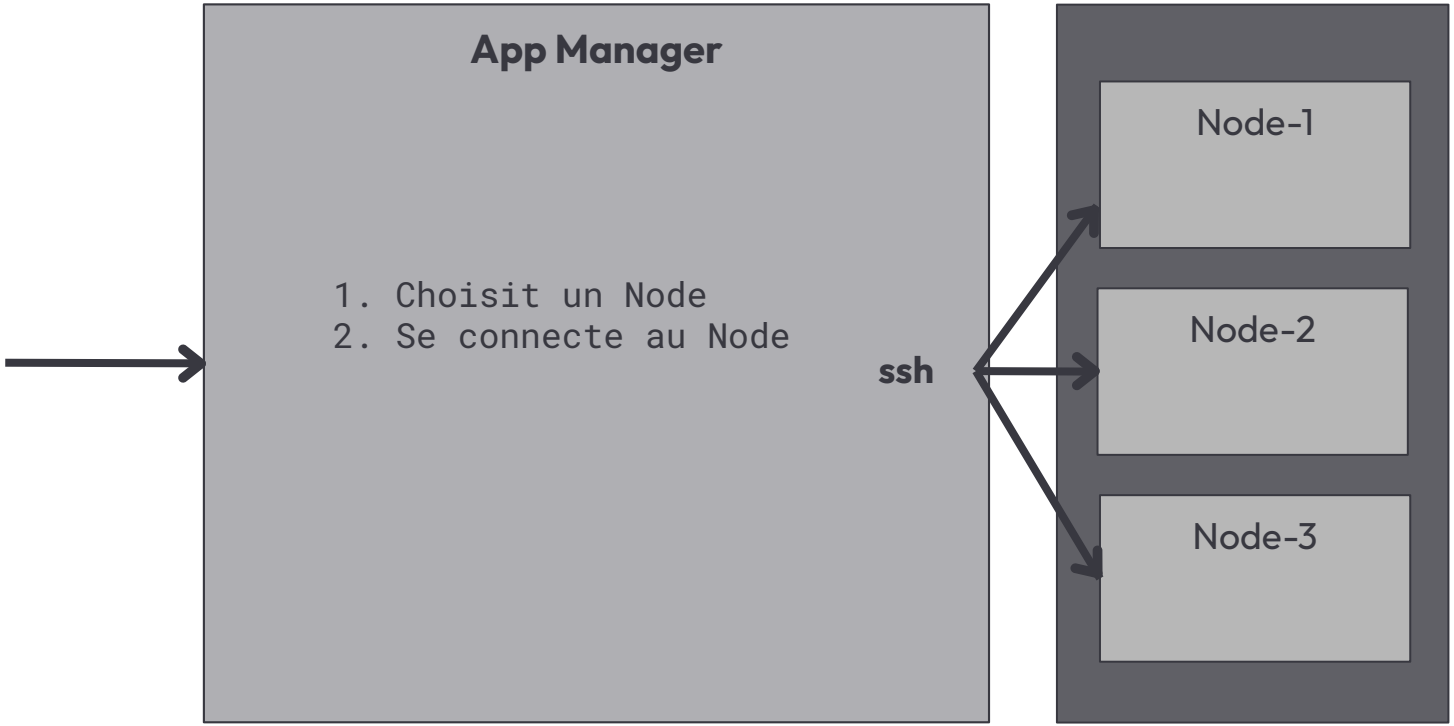




App Manager : version 0

La version la plus simple possible

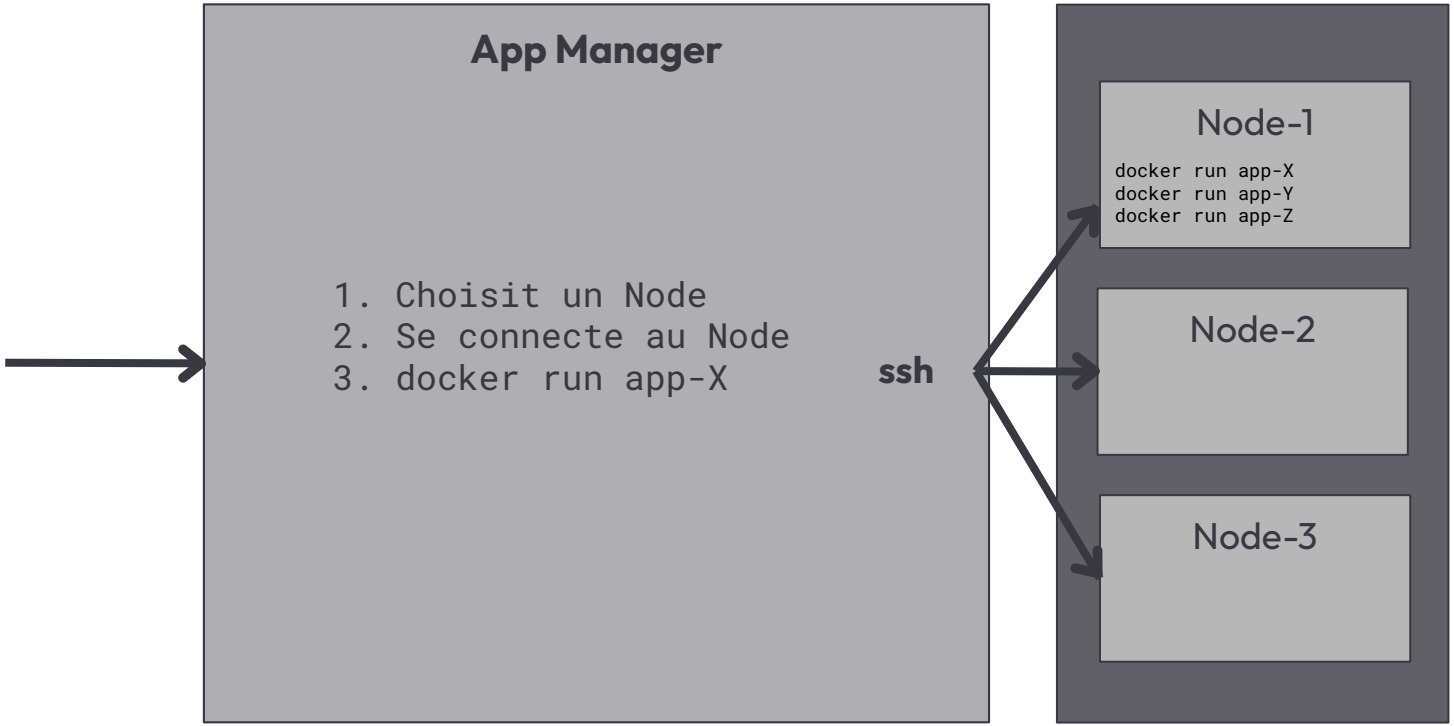
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 0

La version la plus simple possible

```
PUT /app/<name>
{,
"image": "nginx"
}
```

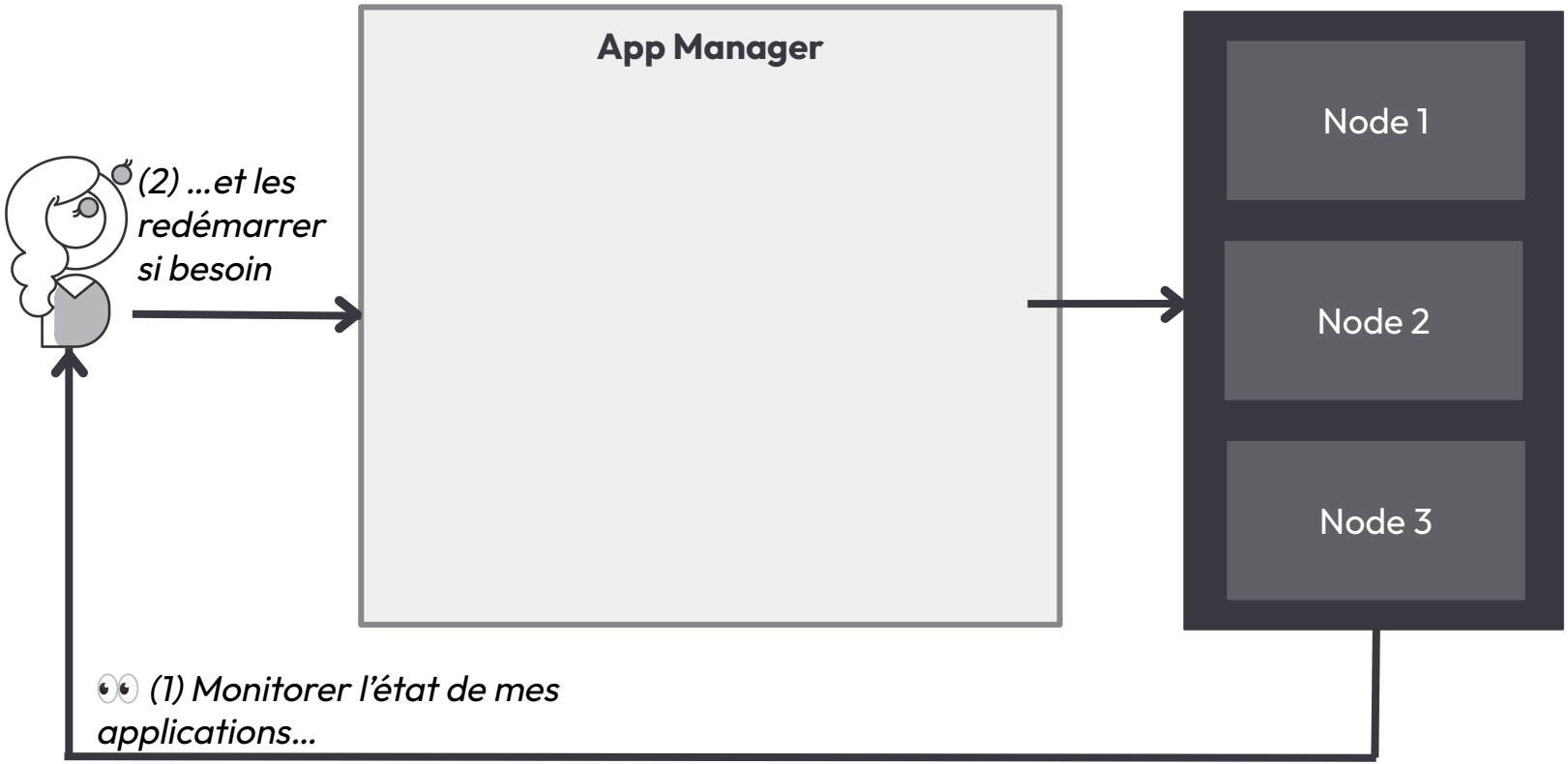


#1 : Déployer oui, mais après ?

Déployer (5 min) 1% du problème	**Maintenir en condition** (8760 heures/an) 99% du problème

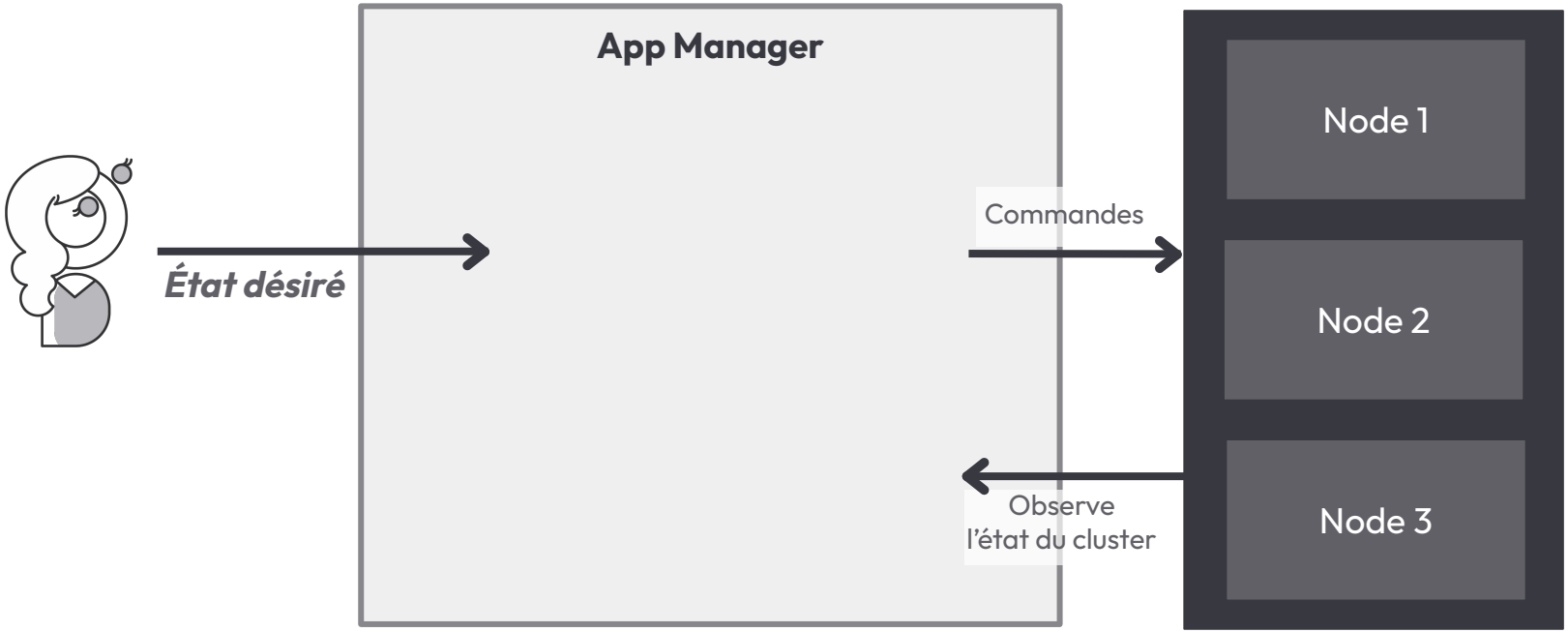
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Déployer oui, mais après ?



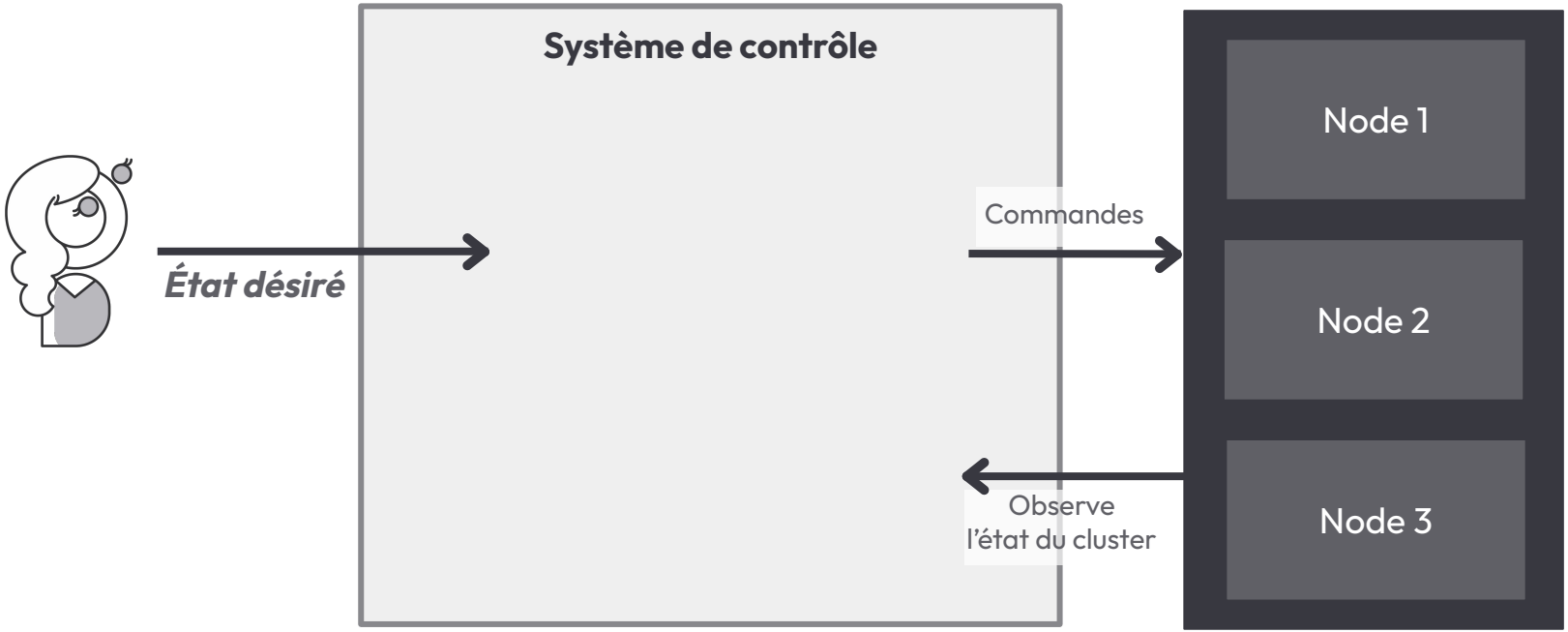
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Déployer oui, mais après ?



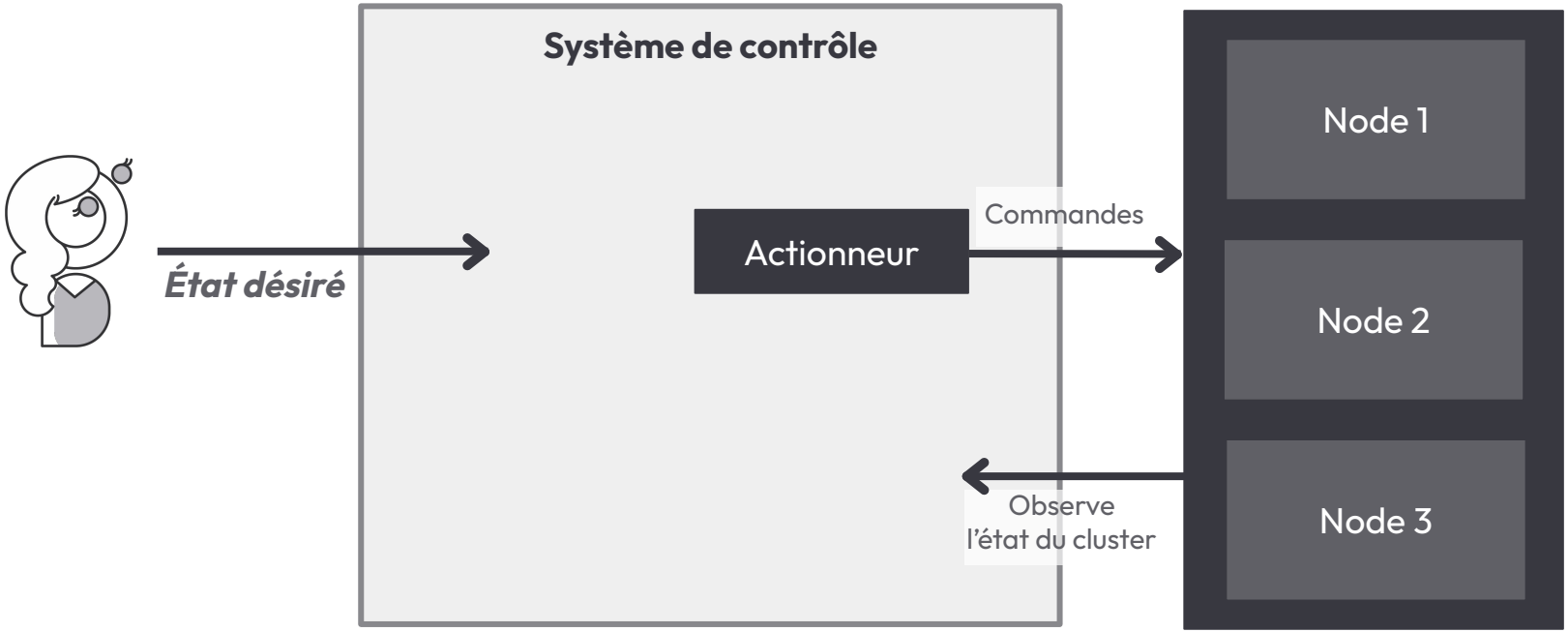
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Déployer oui, mais après ?



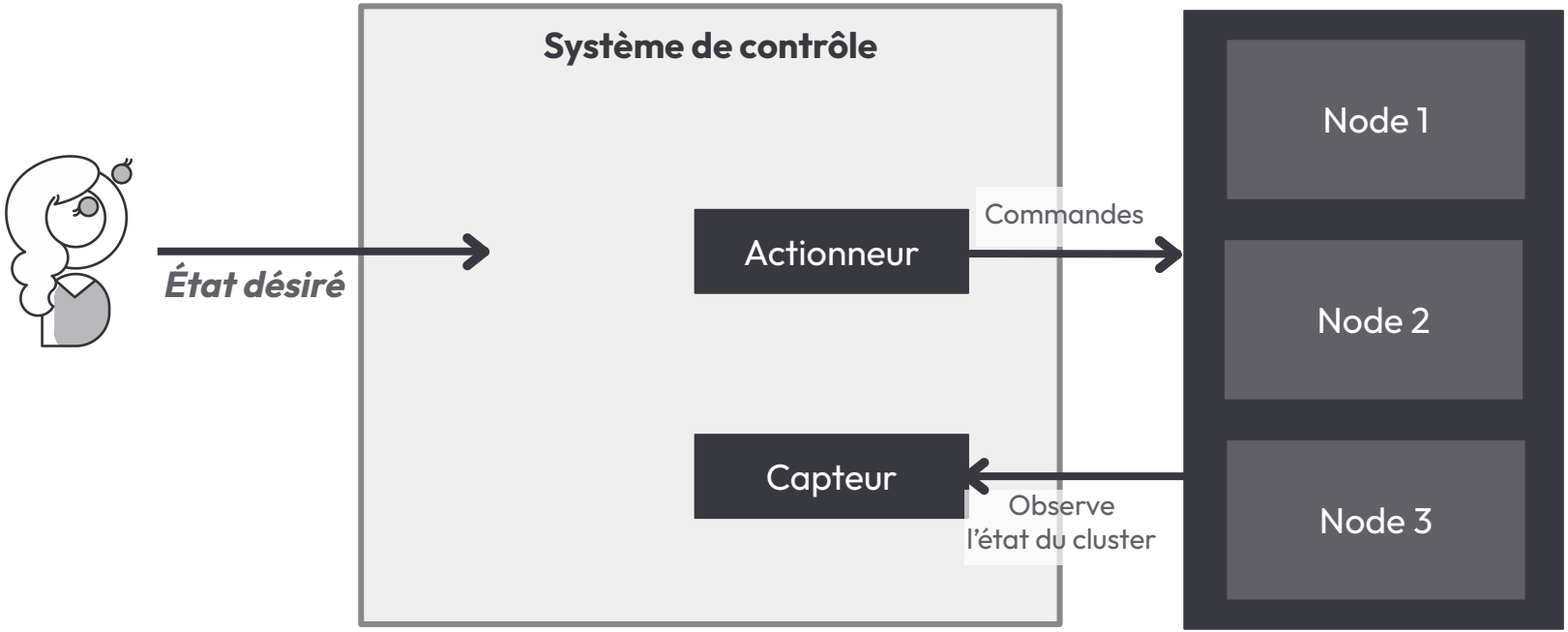
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Déployer oui, mais après ?



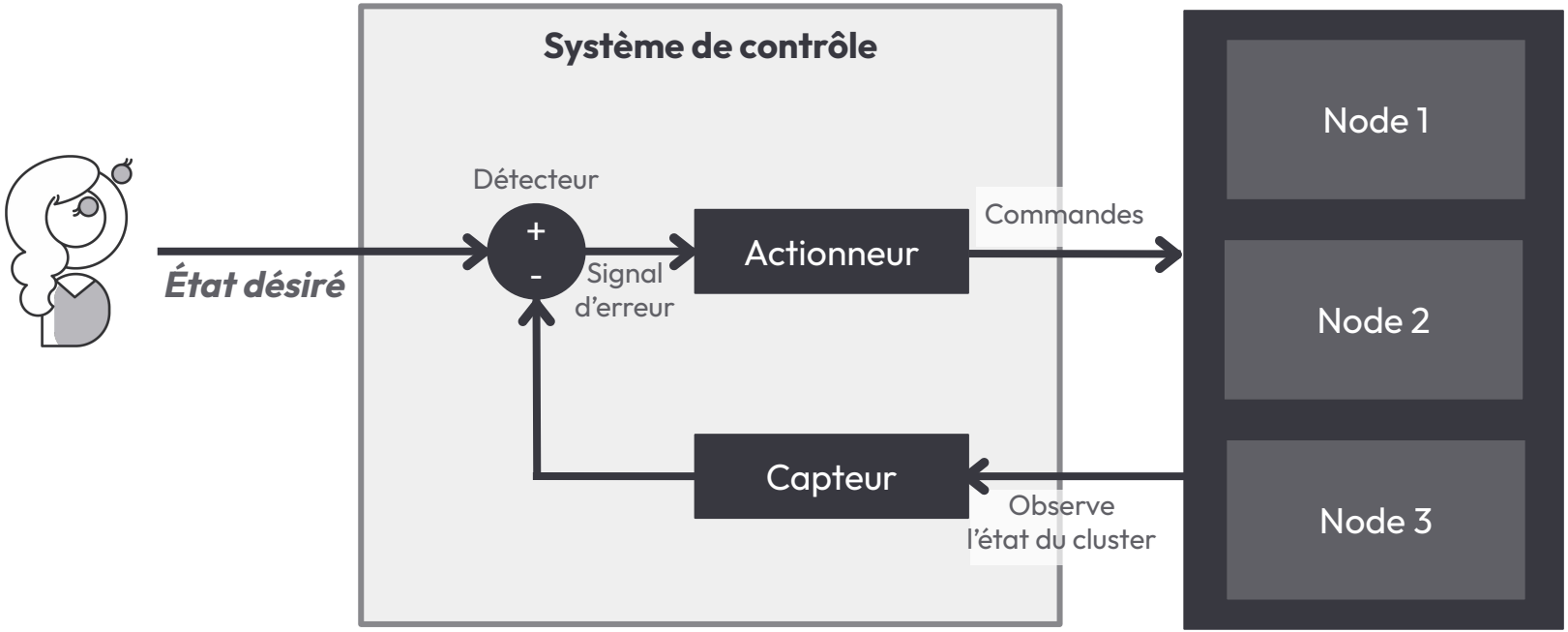
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Déployer oui, mais après ?



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Déployer oui, mais après ?



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Un système de contrôle

2.

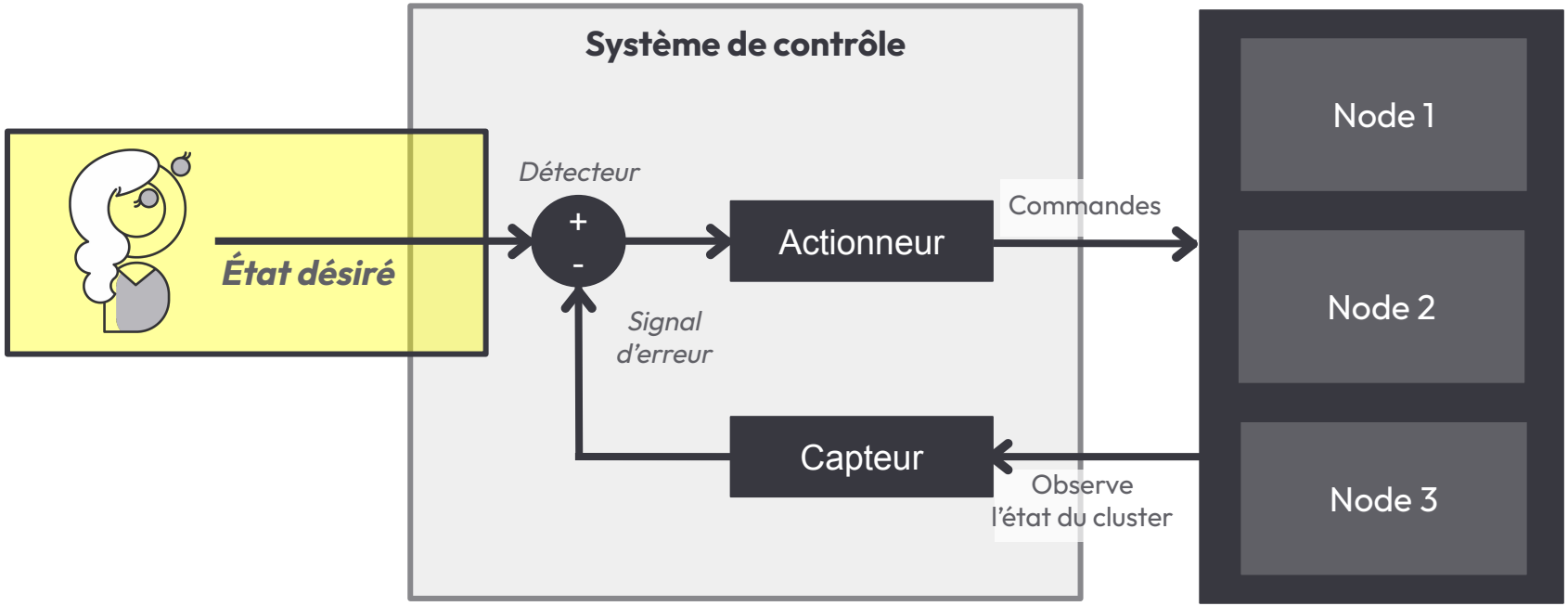
3.

4.

5.

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

2. Une intention plus que des ordres



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

2. Une intention plus que des ordres

Impératif

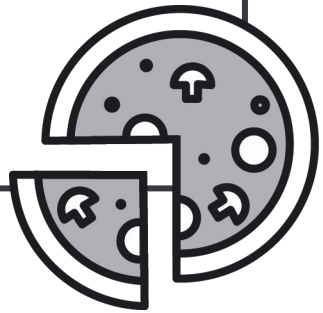
Comment faire

"Prends de la farine, puis ajoute de l'eau, puis pétrie, puis ajoute de la sauce tomate..."

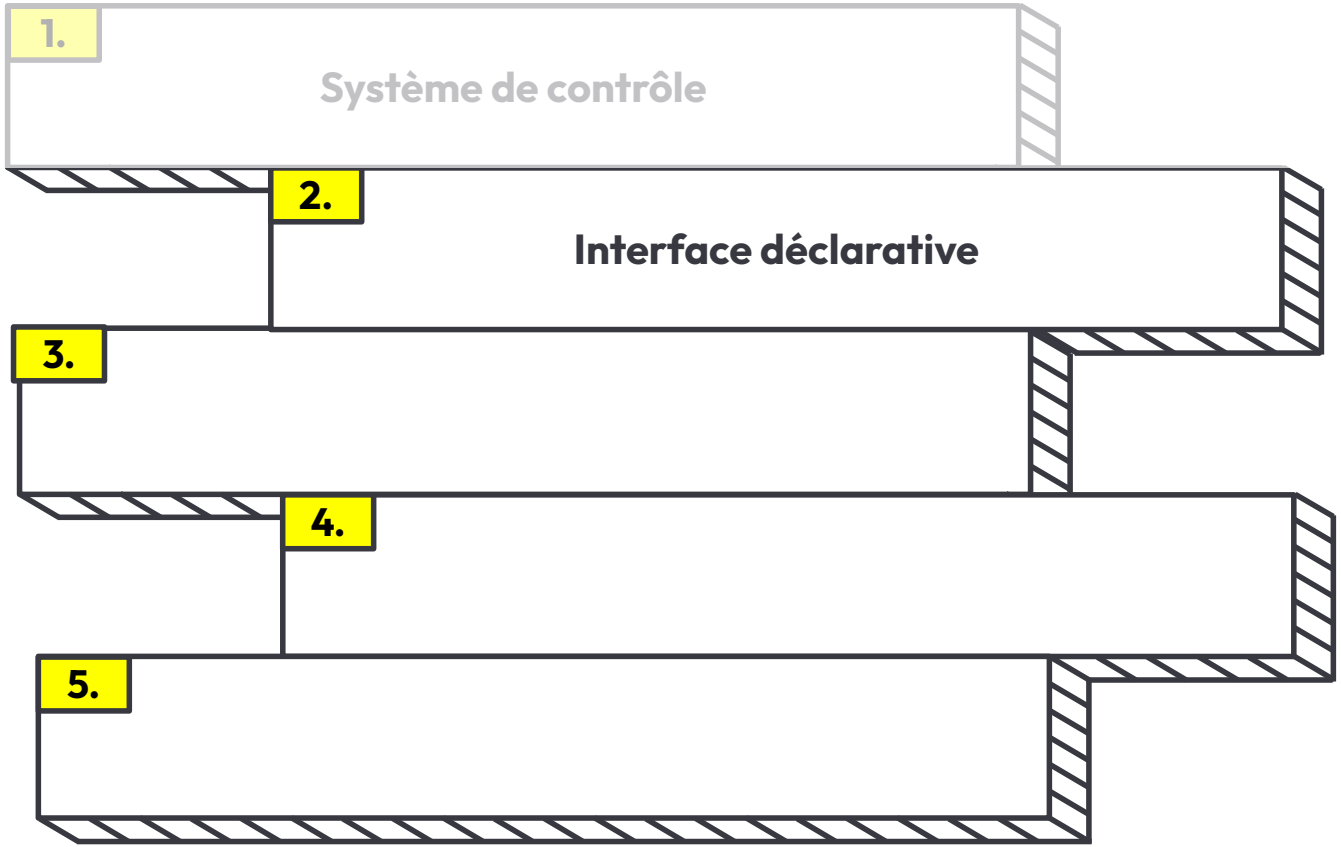
Déclaratif

Résultat attendu

"Je veux une pizza"

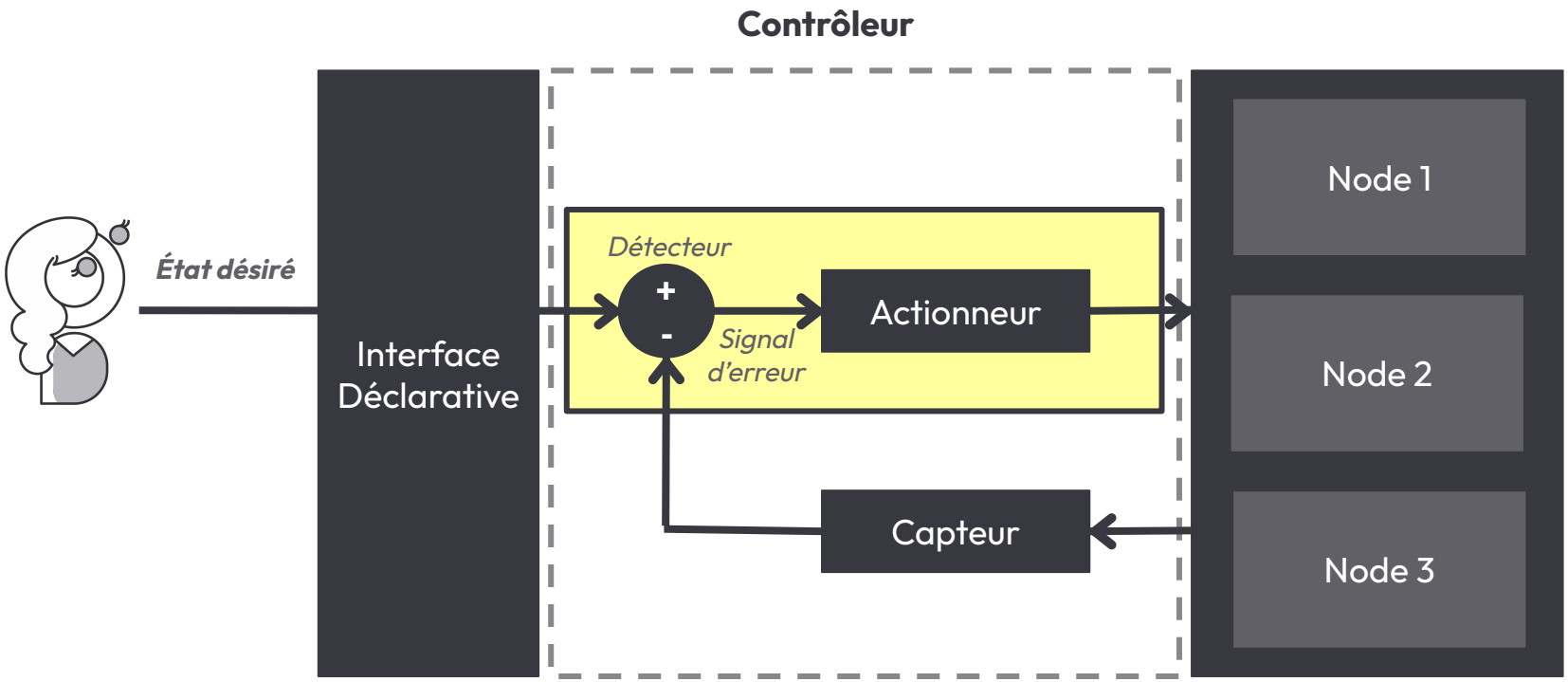


LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

3. Événement ou État ?

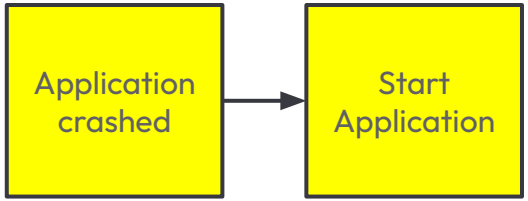


LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

3. Événement ou État ?

Orienté événement - Edge Trigger

Déclenché sur un évènement



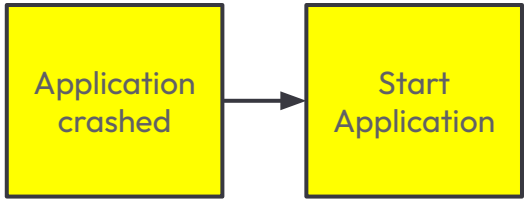

```
while(true):
    waitForEvent() // bloquant
    act()
```

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

3. Événement ou État ?

Orienté événement - Edge Trigger

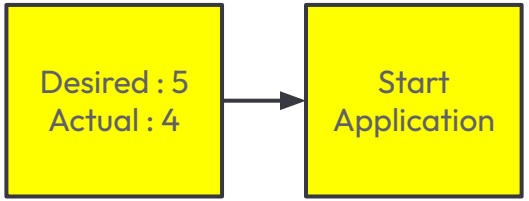
Déclenché sur un évènement




```
while(true):
    waitForEvent() // bloquant
    act()
```

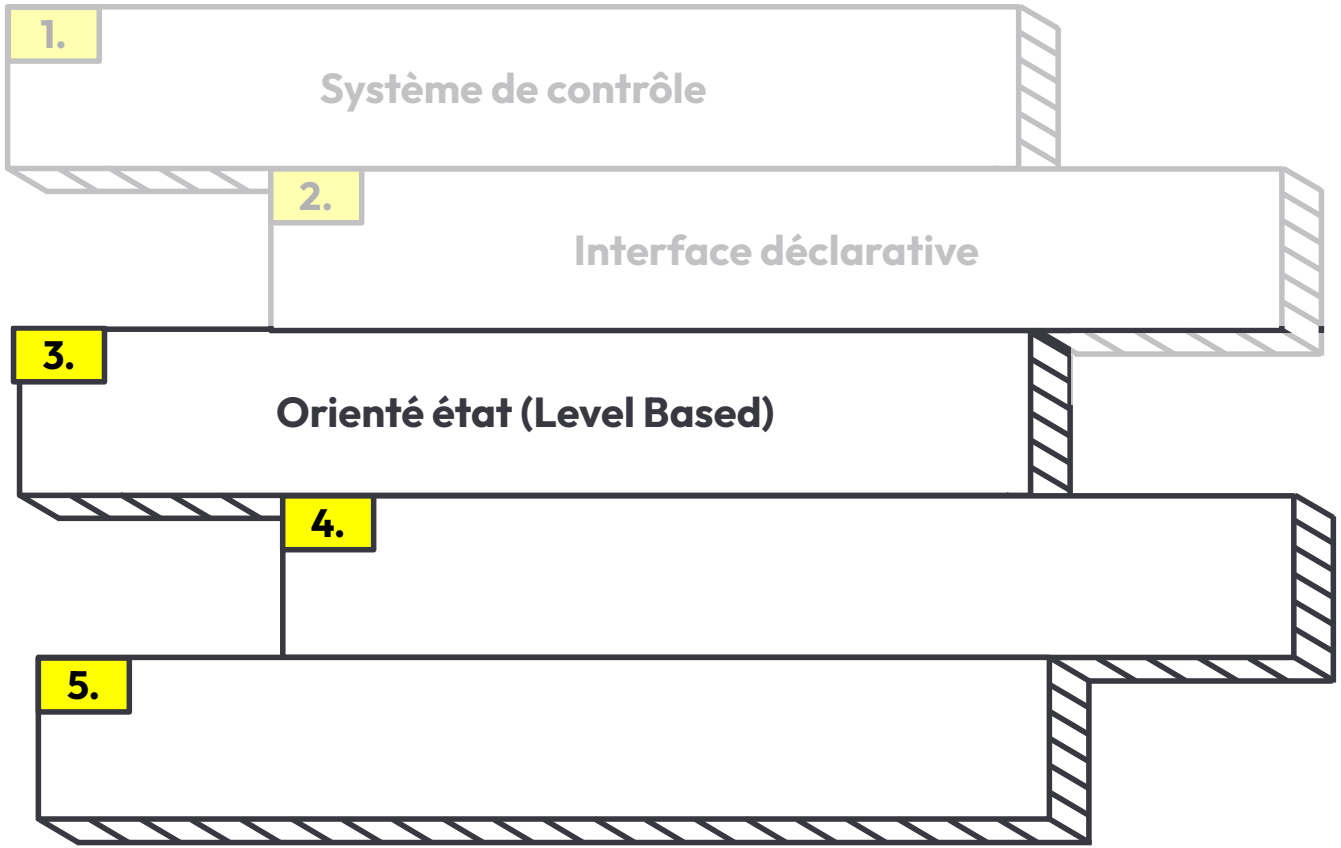
Orienté état - Level Based

Comparaison continuelle entre état désiré et état actuel




```
while(true):
    getEtatActuel()
    getEtatDesire()
    if(actuel != desire)
       act()
```

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

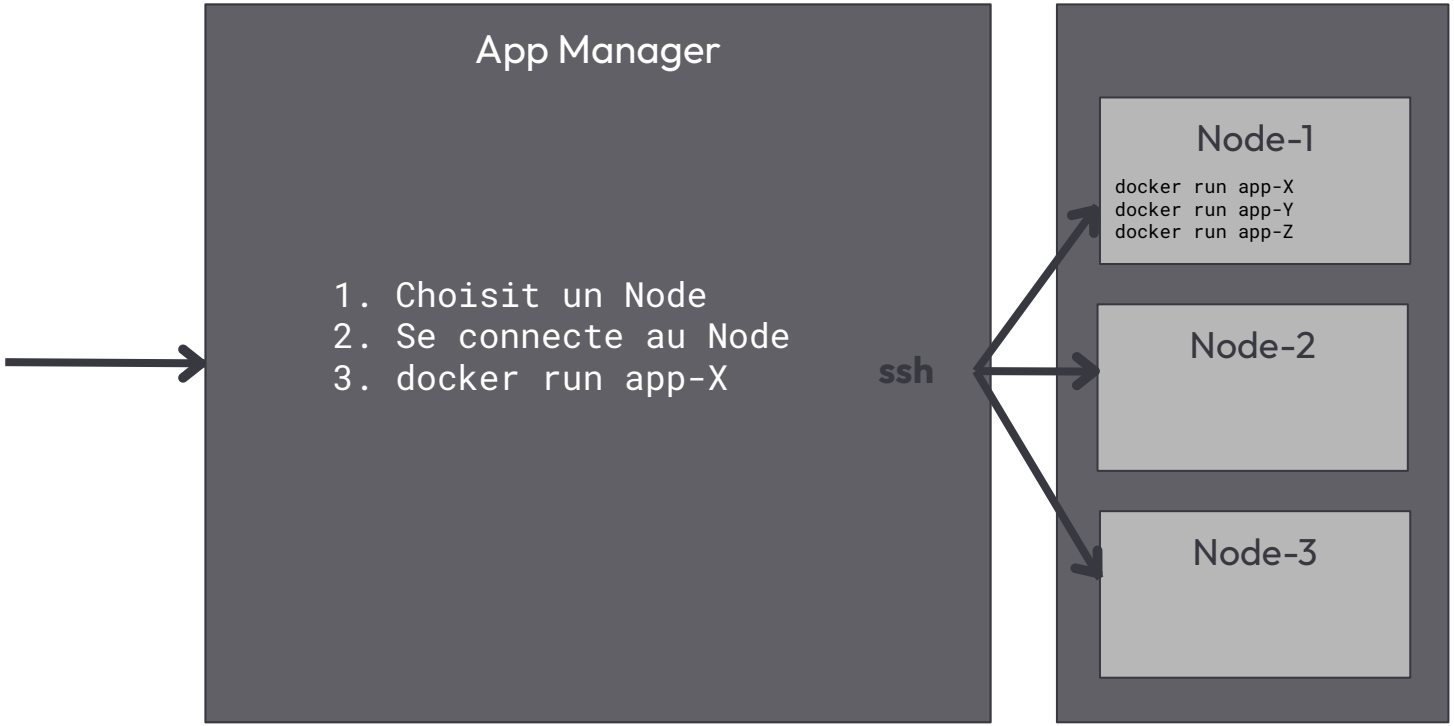


LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

App Manager : version 1

Appliquons nos 3 premiers principes

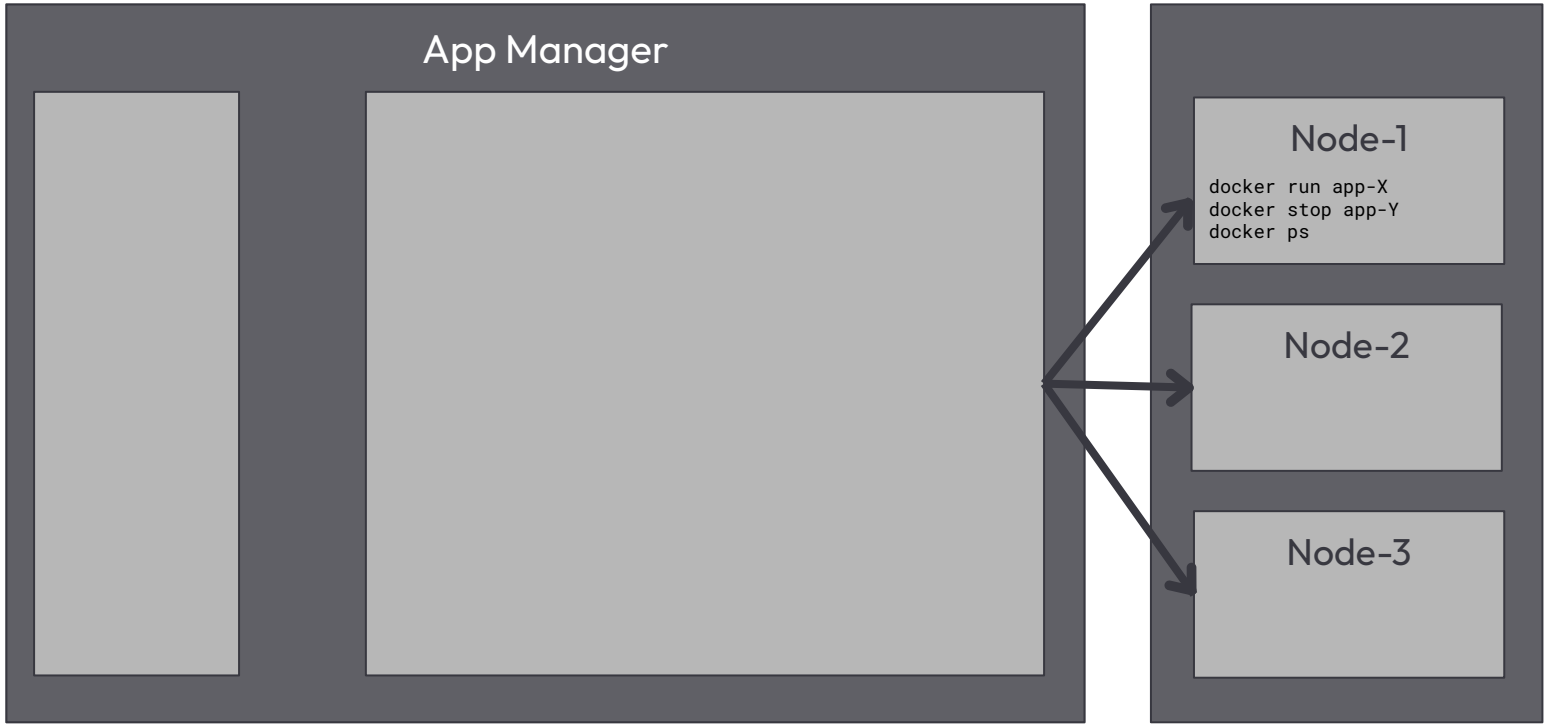
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

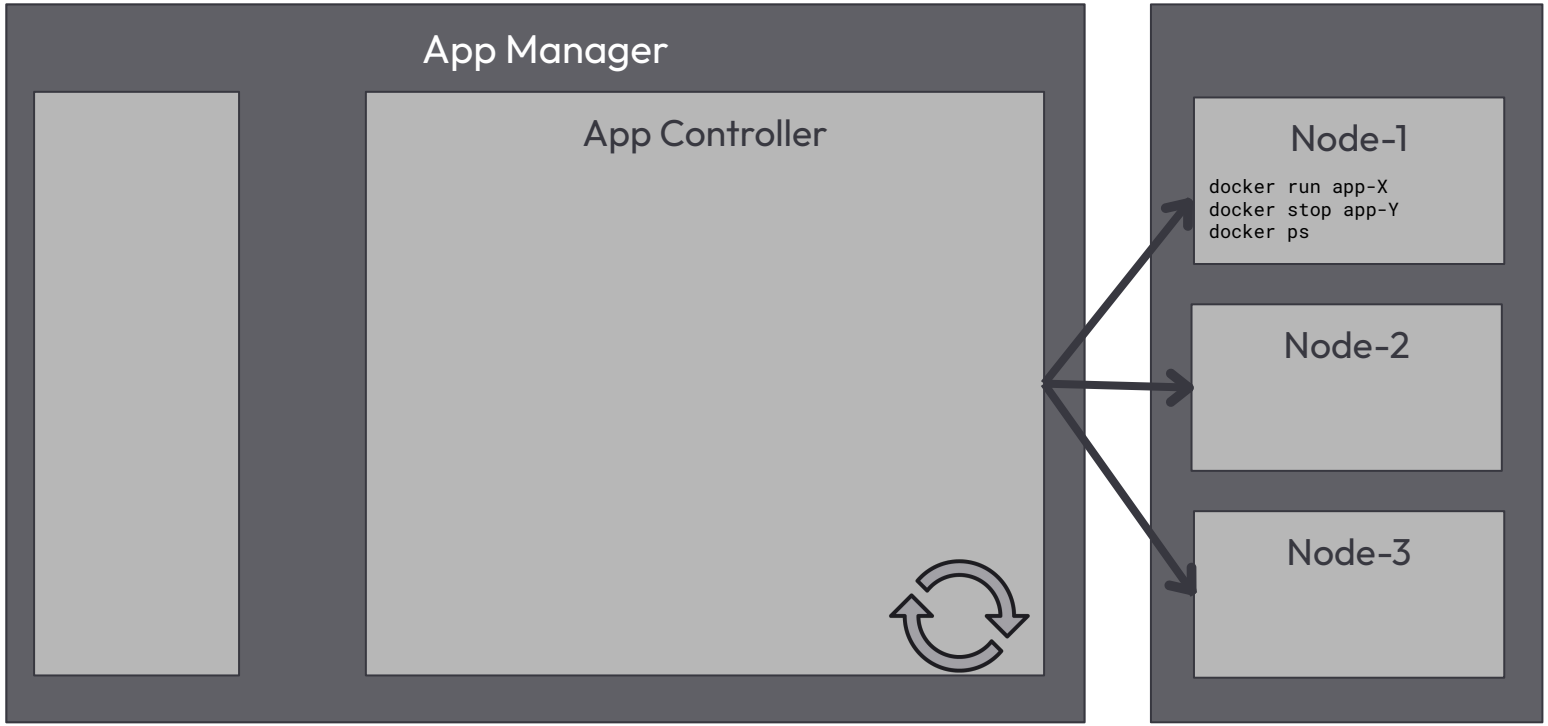
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

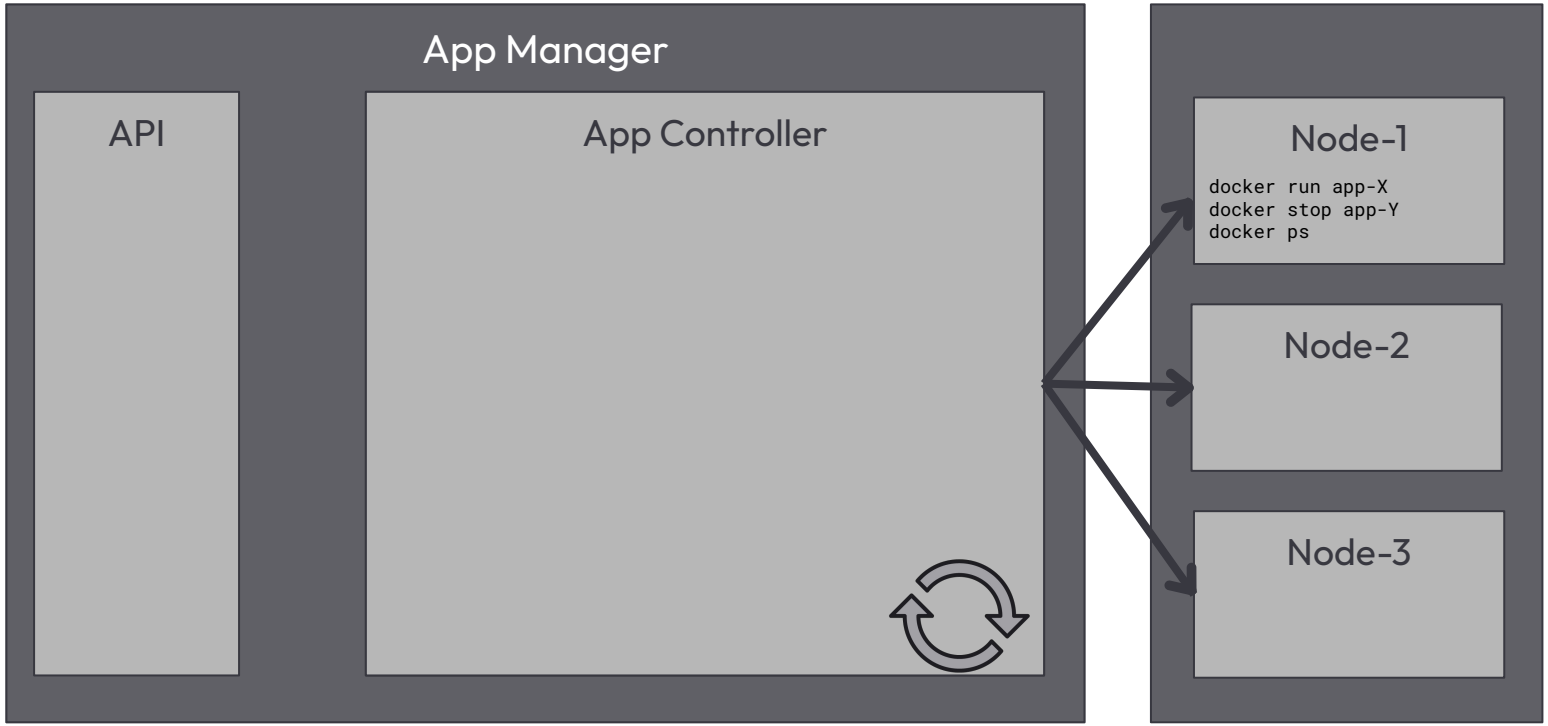
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

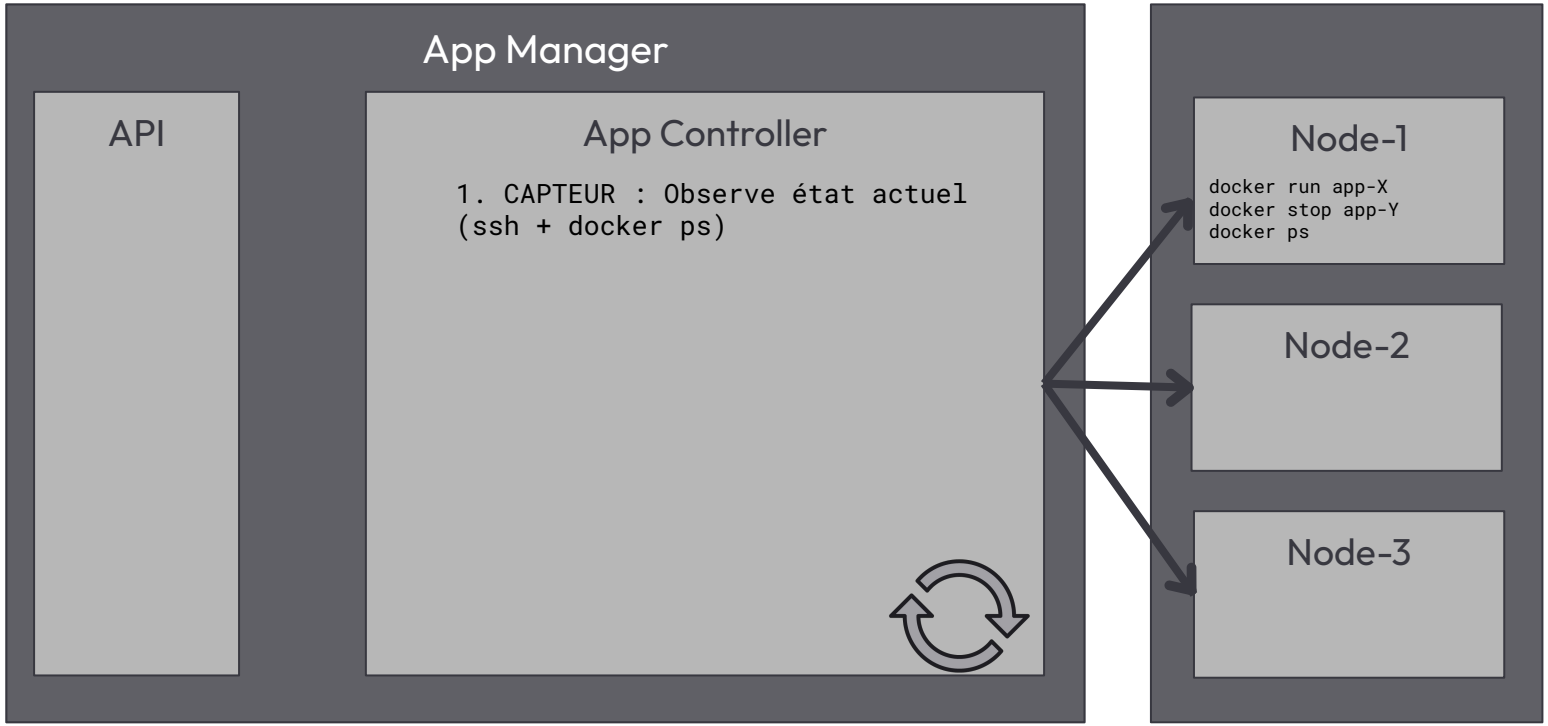
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

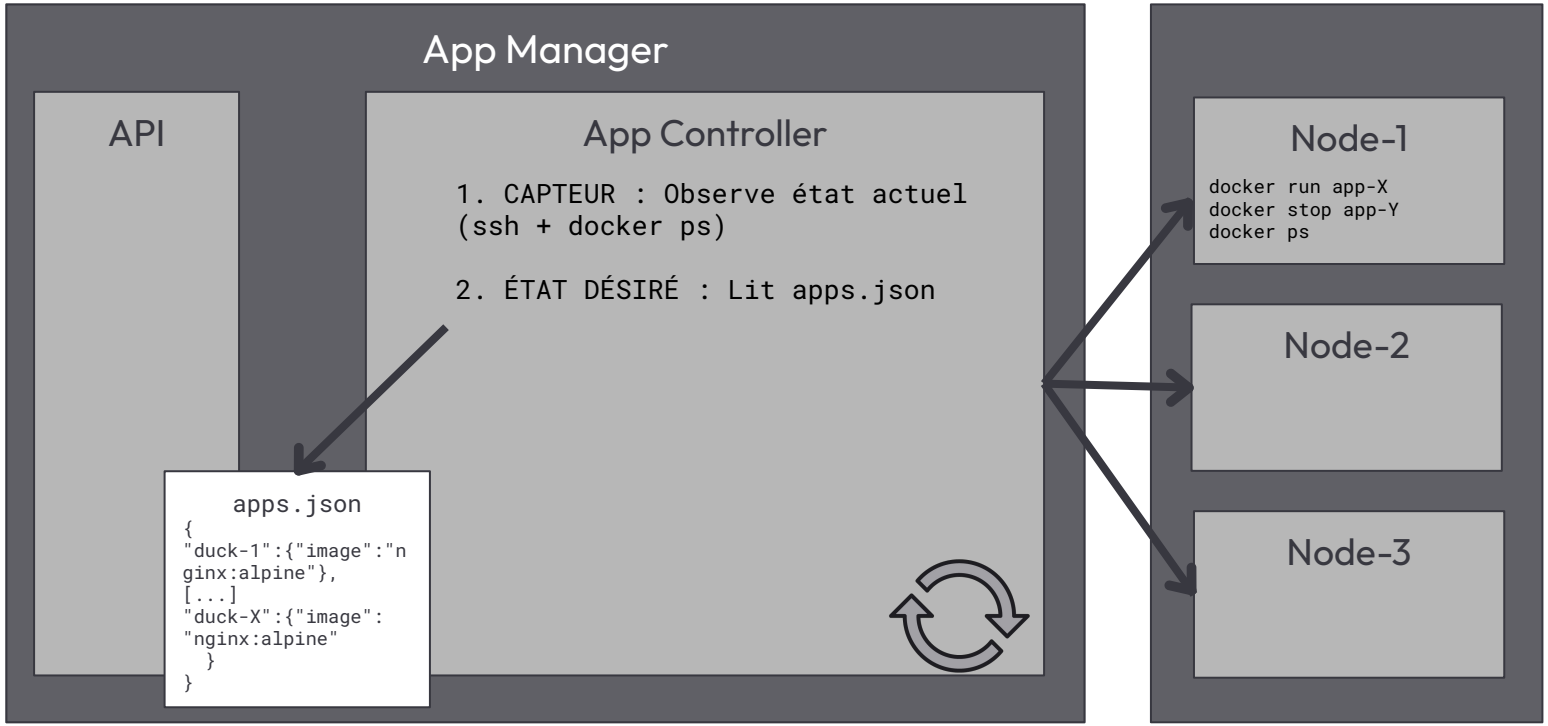
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

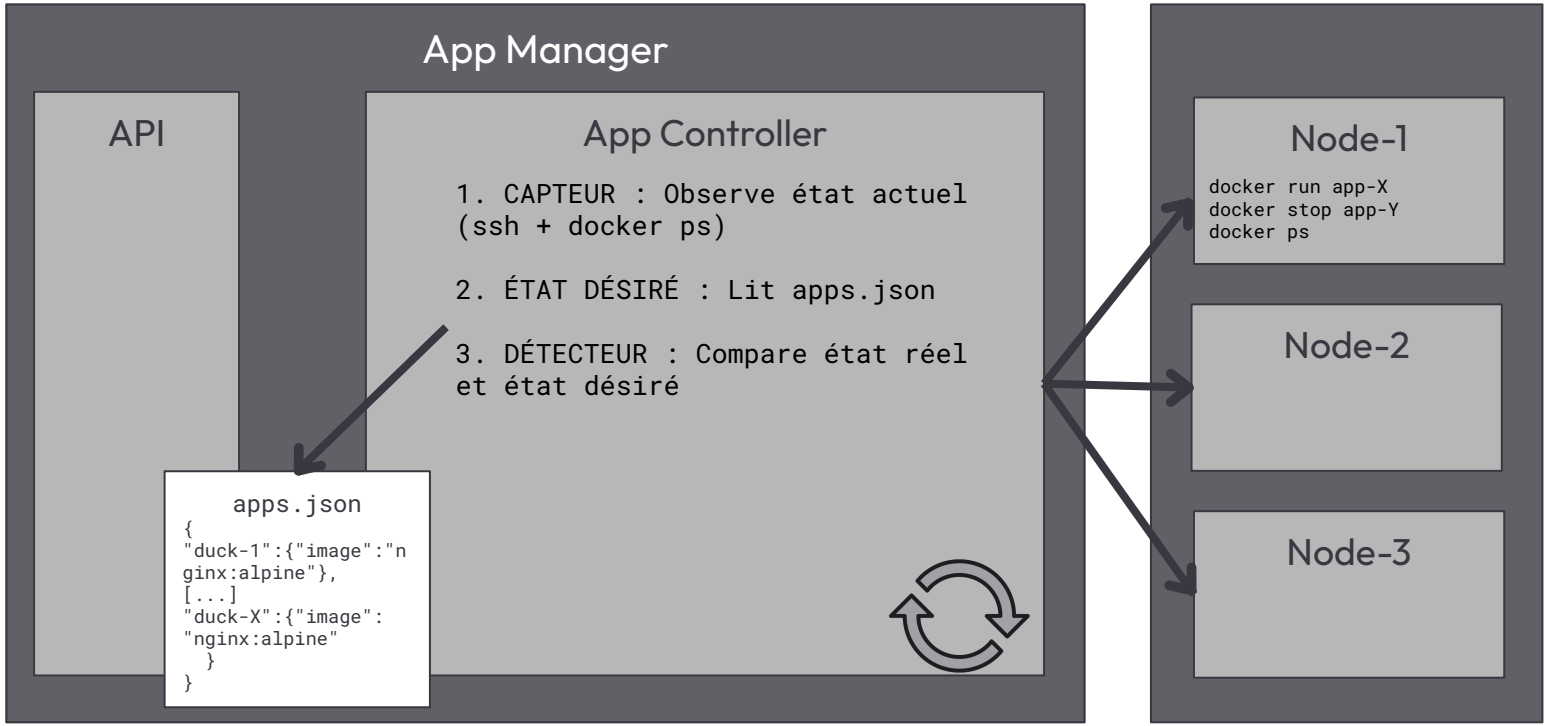
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

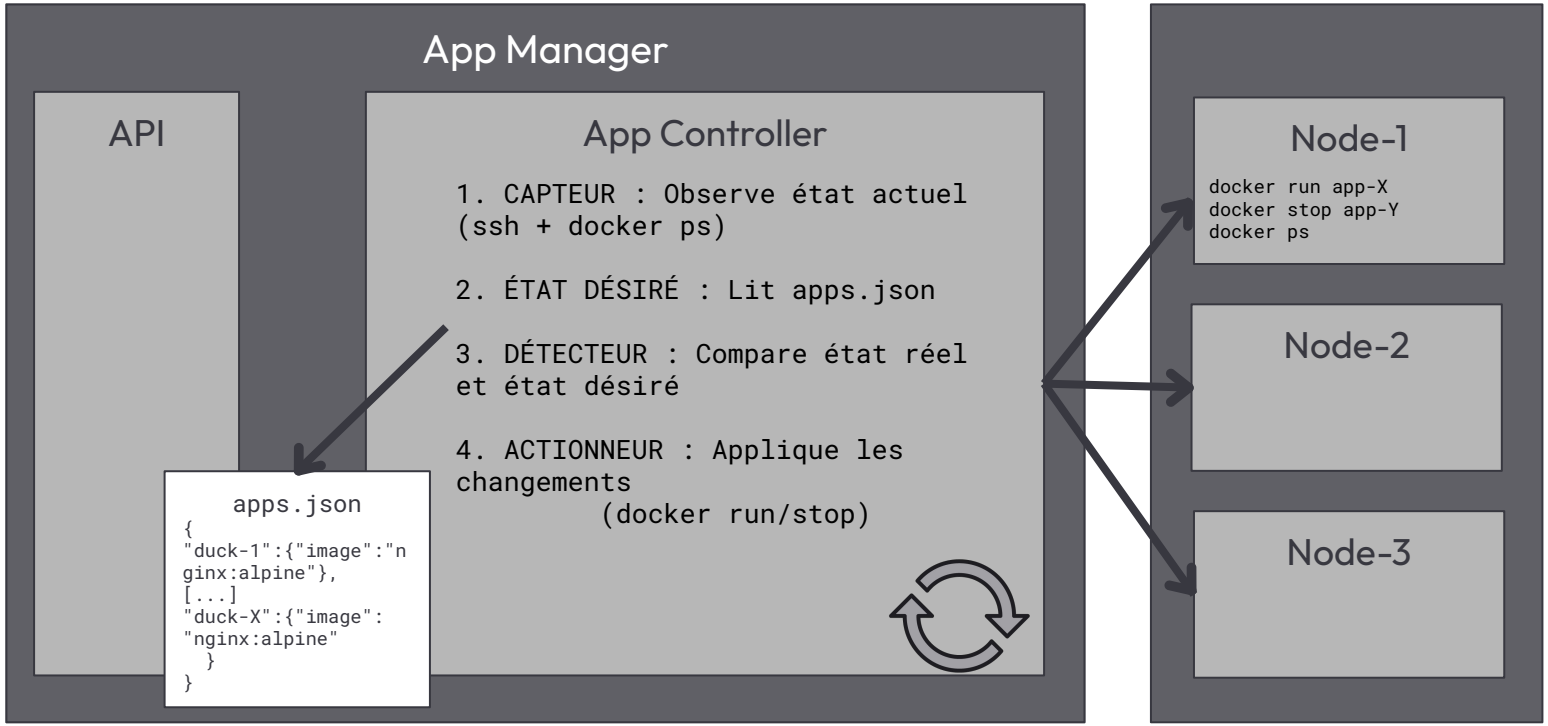
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

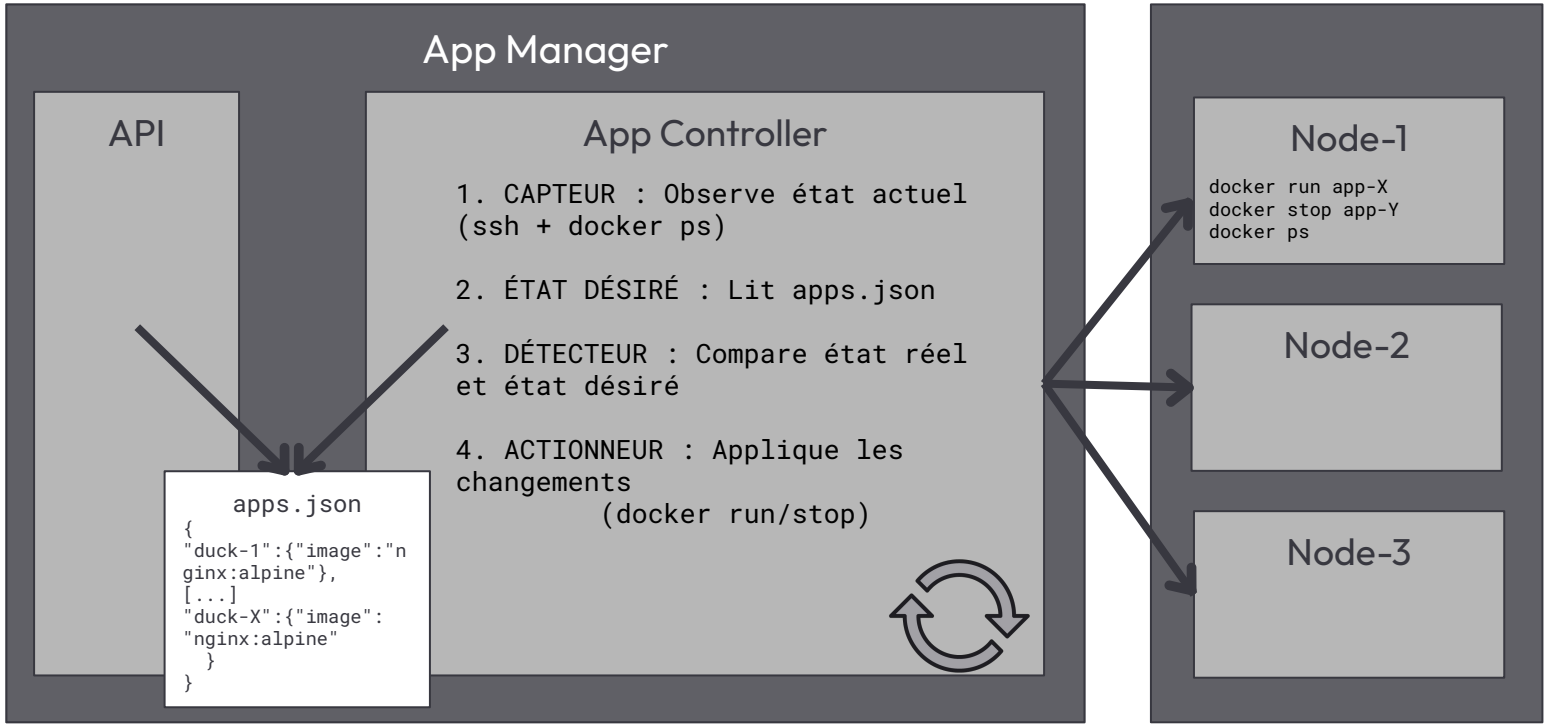
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version 1

Appliquons nos 3 premiers principes

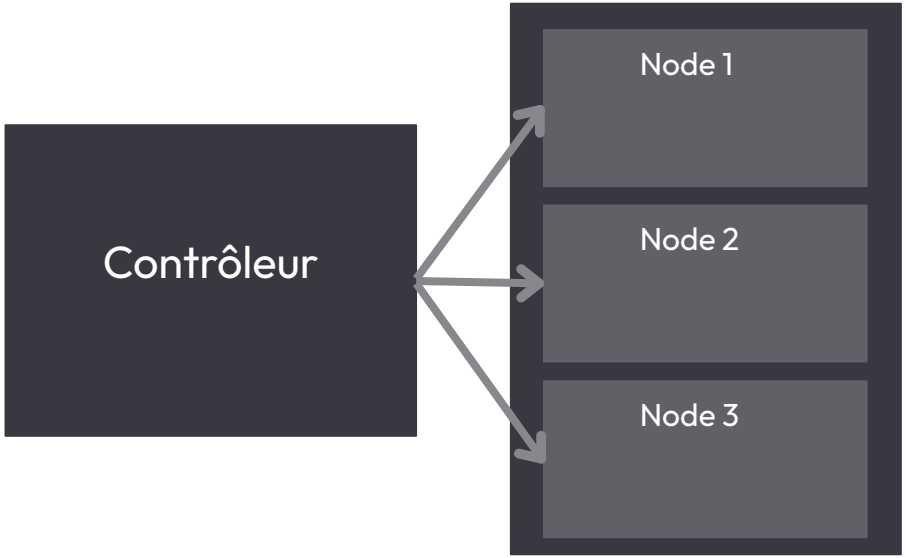
```
PUT /app/<name>
{,
"image": "nginx"
}
```



4. “Do one thing and do it well”

Un seul contrôleur responsable de

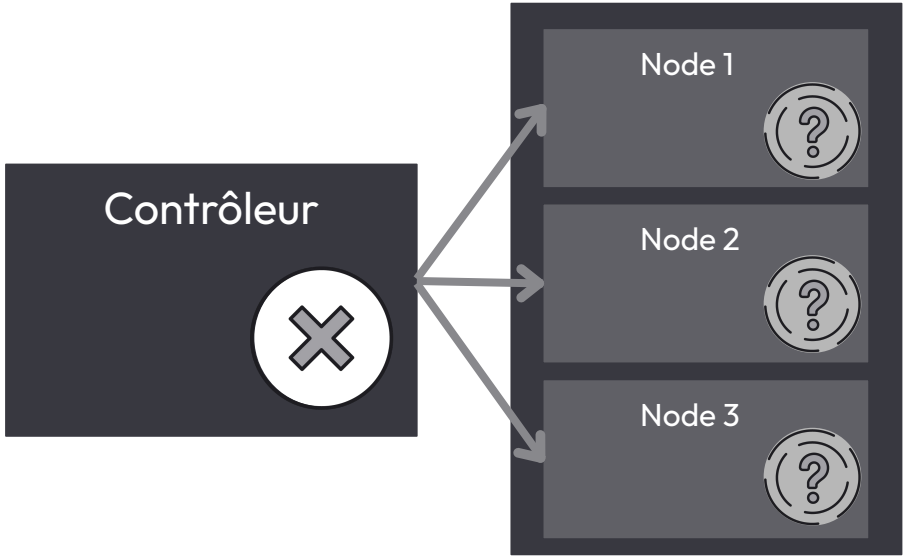
TOUT



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

4. “Do one thing and do it well”

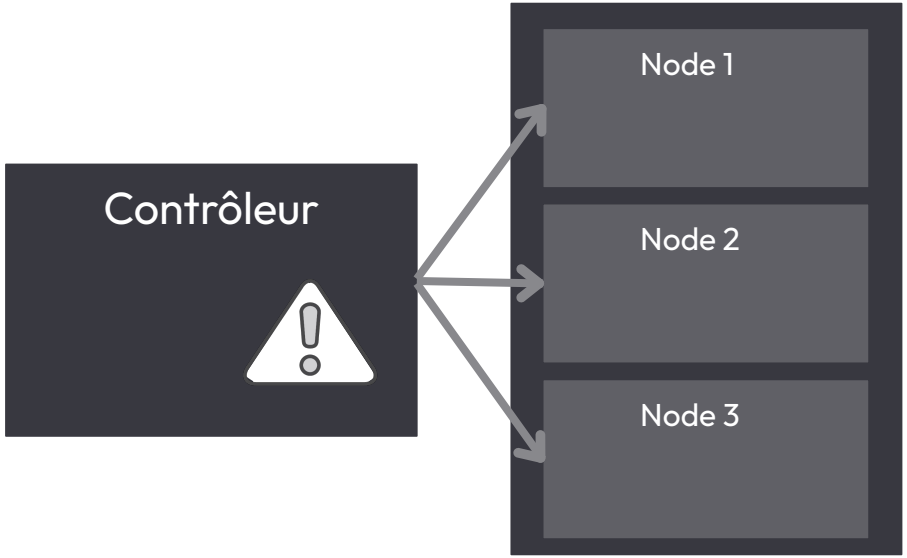
Si le contrôleur tombe, le cluster **dérive**



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

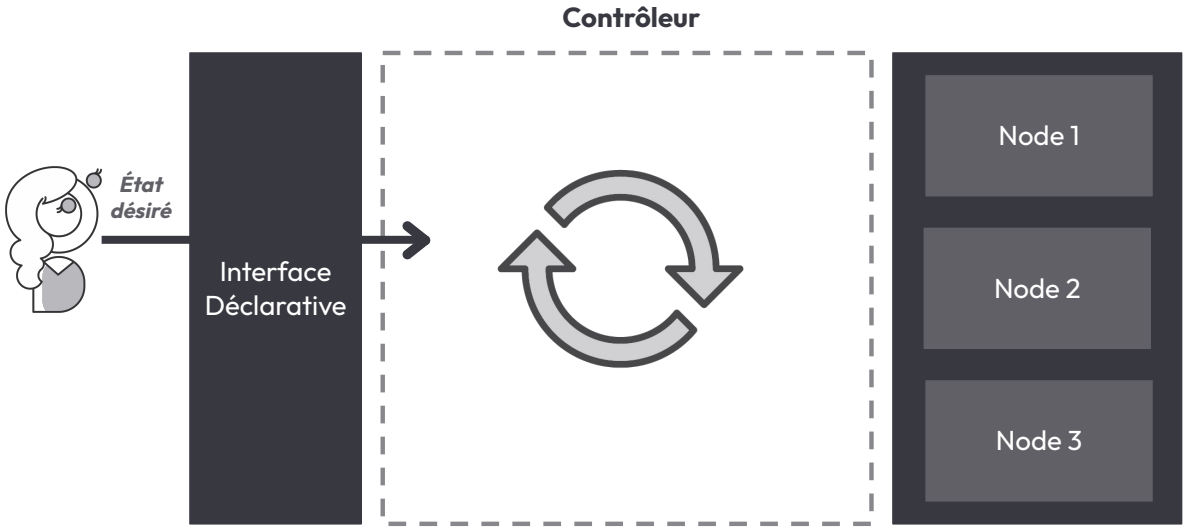
4. “Do one thing and do it well”

Au moindre cas non géré, **toute la boucle plante**



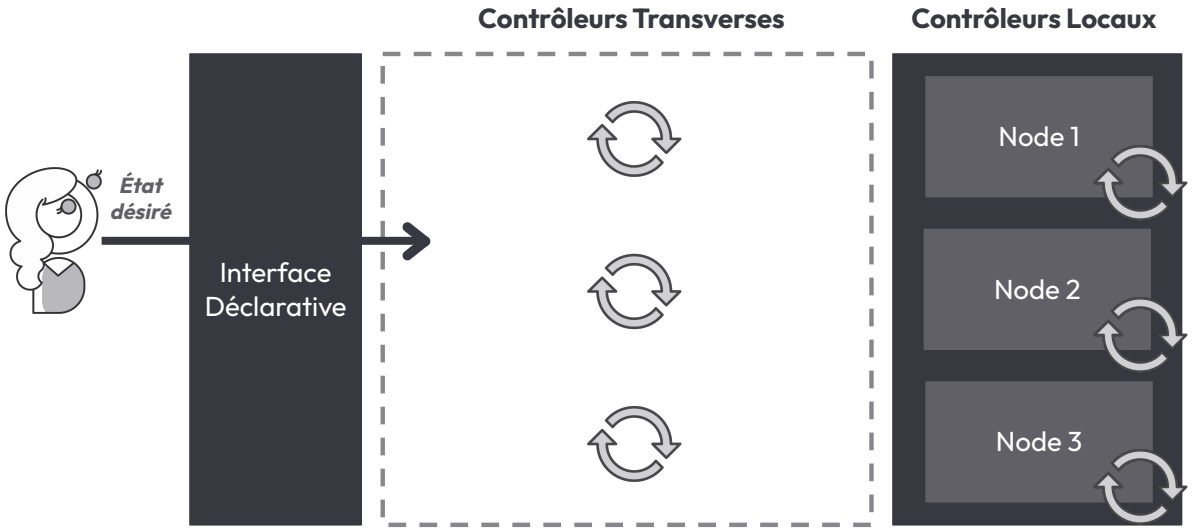
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

4. “Do one thing and do it well”



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

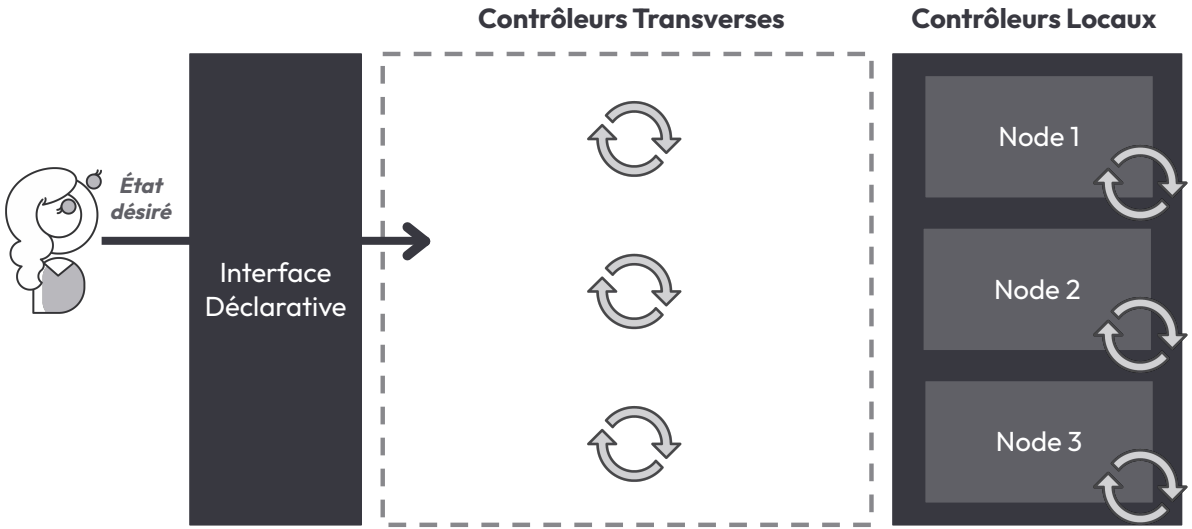
4. “Do one thing and do it well”



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

4. “Do one thing and do it well”

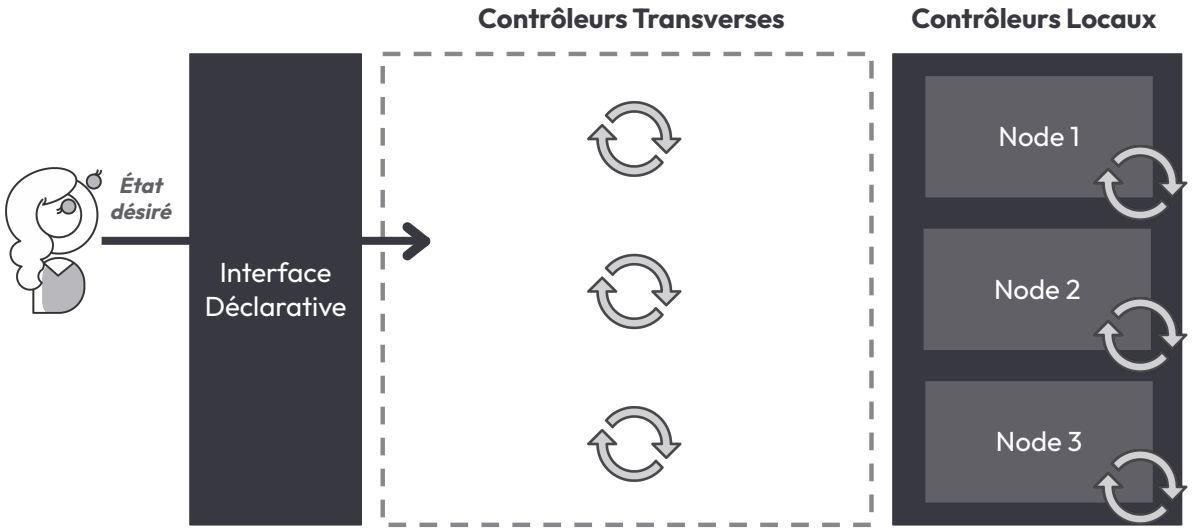
Au plus proche du système contrôlé



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

4. “Do one thing and do it well”

- Au plus proche du système contrôlé
- Responsabilité claire et limitée



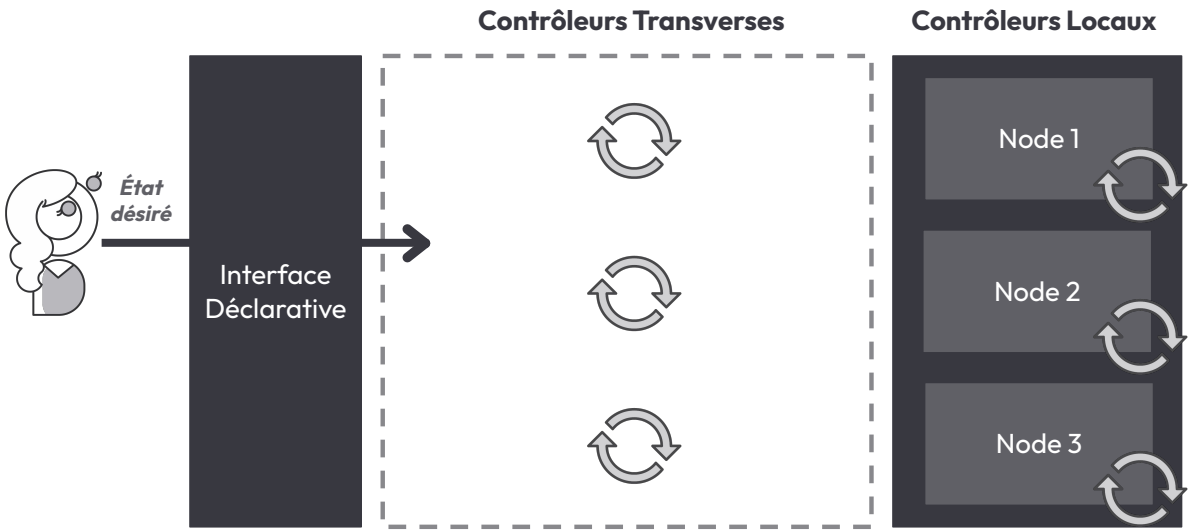
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

4. “Do one thing and do it well”

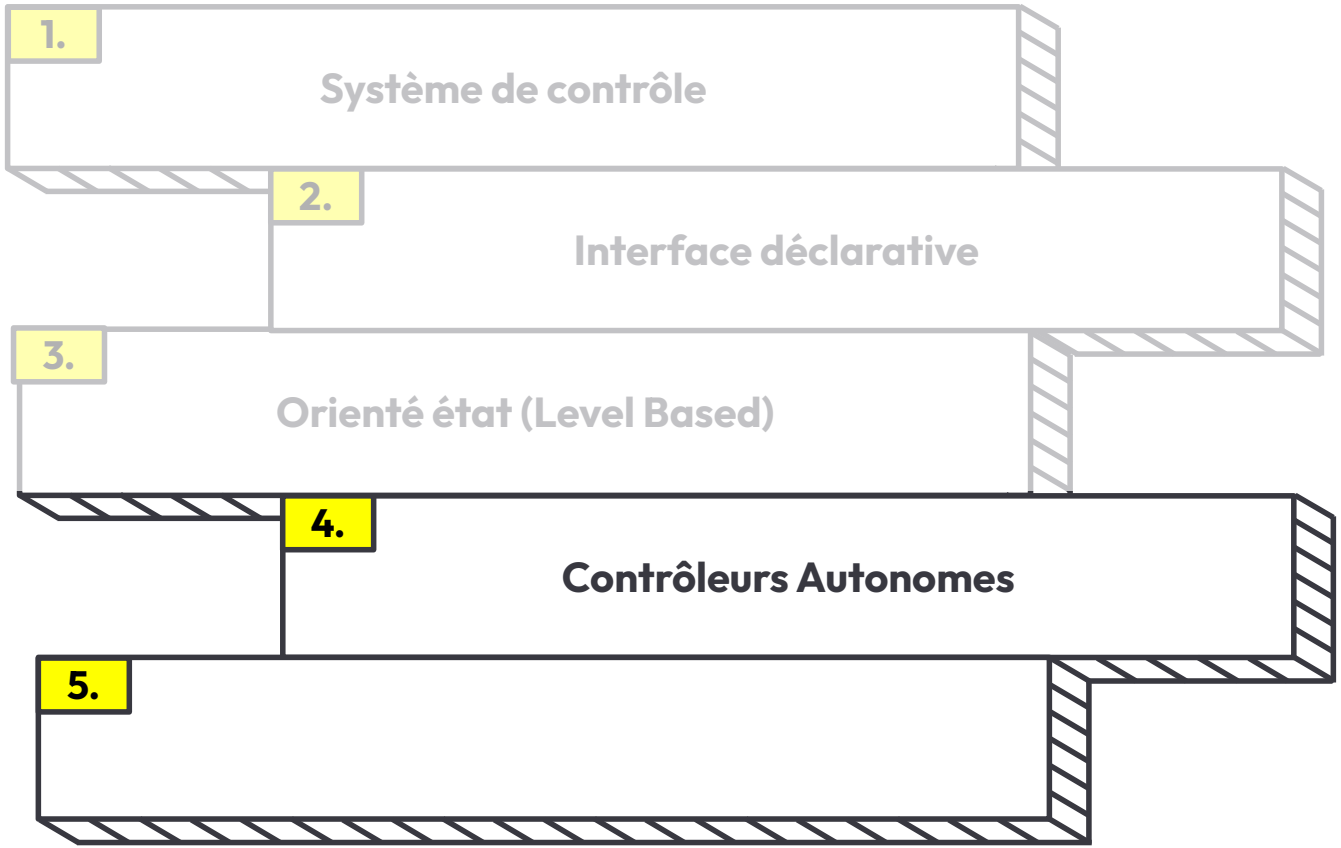
Au plus proche du système contrôlé

Responsabilité claire et limitée

Peuvent fonctionner en autonomie



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

5. Créer un consensus

01. Contrôleurs locaux

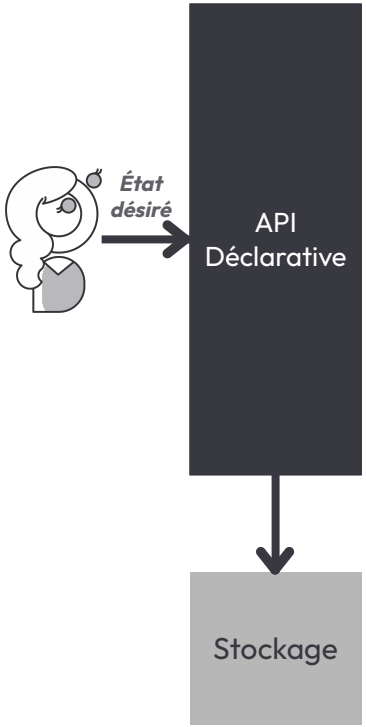
- Executent les commandes
- N'implémente pas de logique transverse

02. Contrôleurs Transverses

- Implémentent la logique de coordination
- N'exécutent pas de commandes

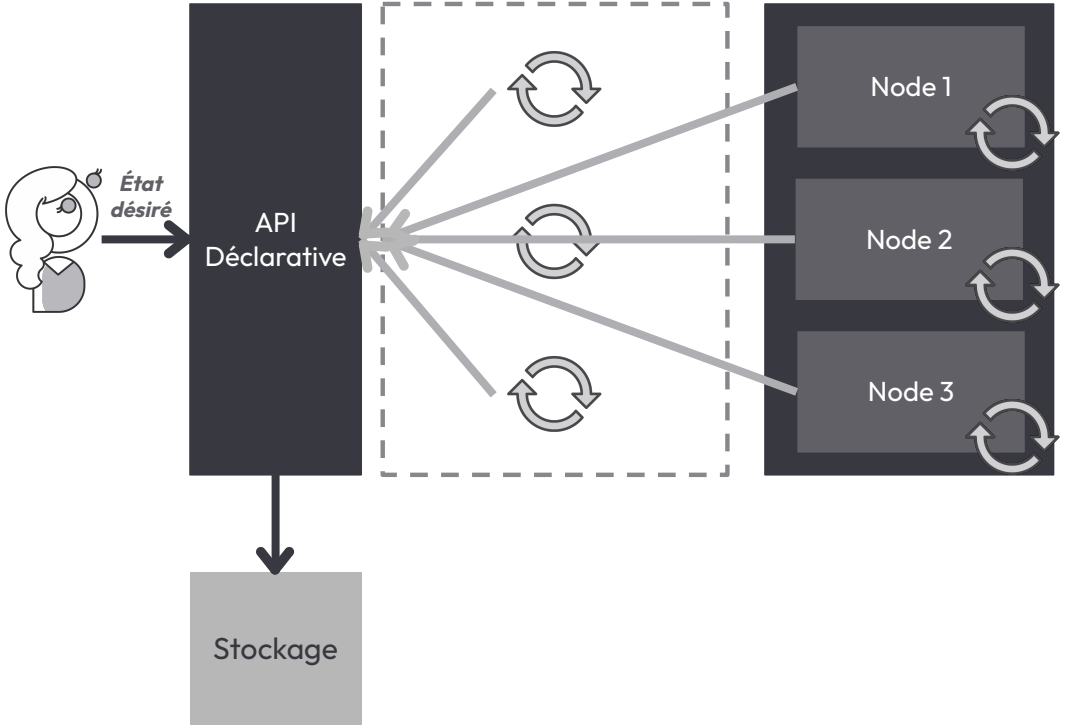
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

5. Créer un consensus



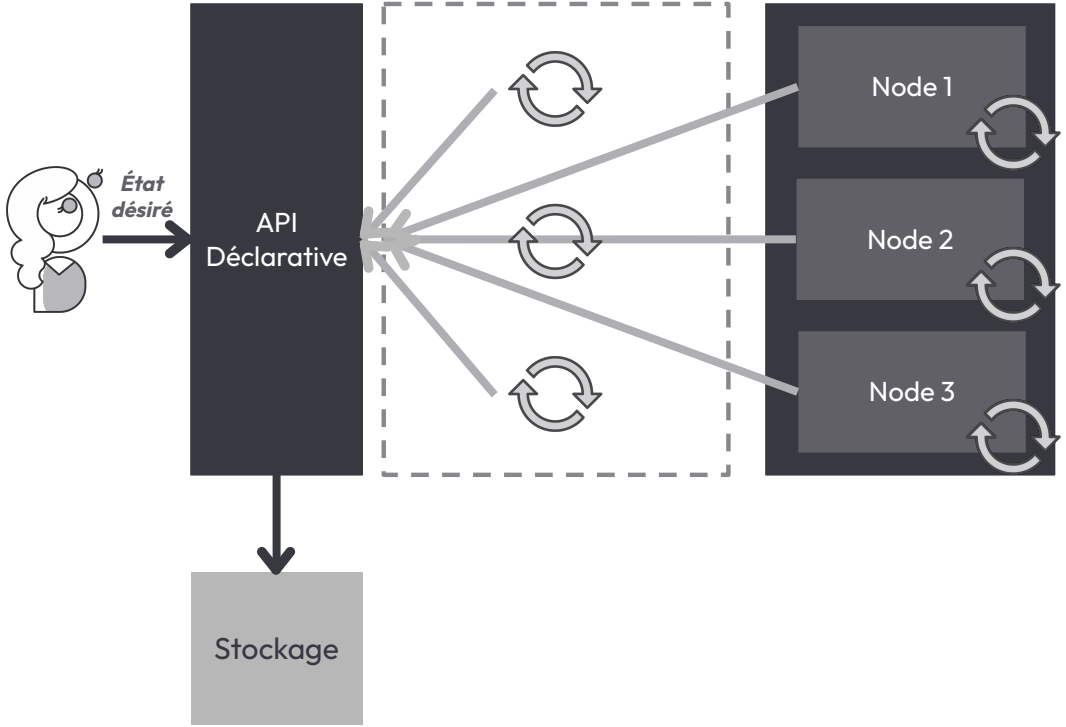
LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

5. Créer un consensus



LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

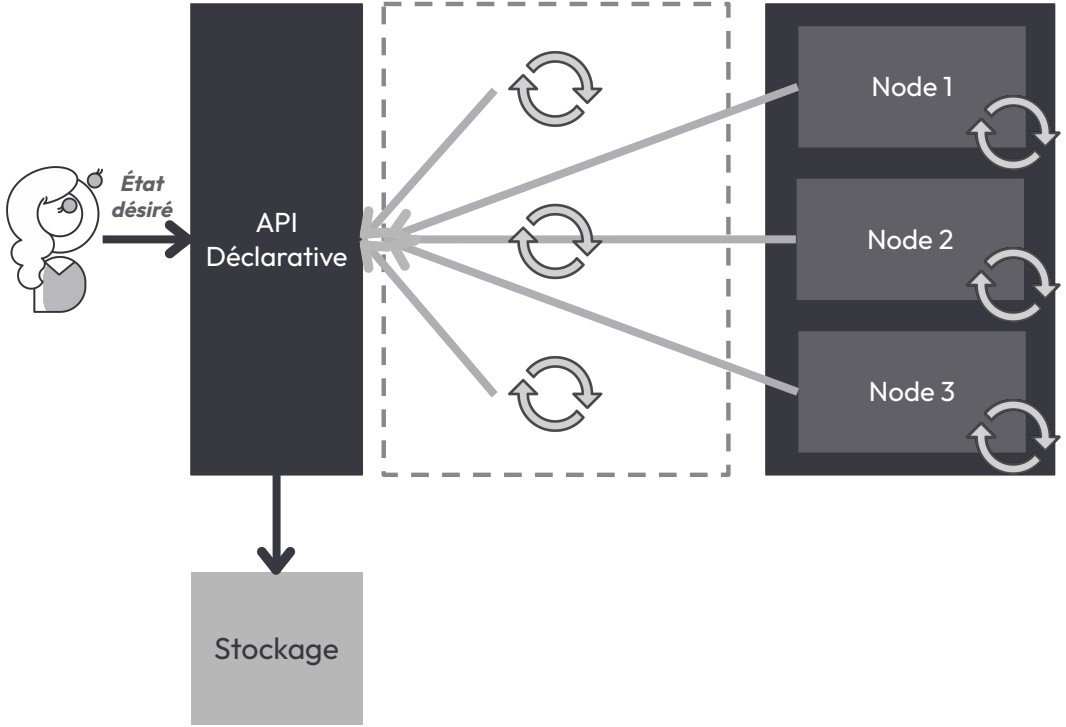
5. Créer un consensus



01. API Unique et Centralisée

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

5. Créer un consensus



01. API Unique et Centralisée

02. API Transparente

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

1. Système de contrôle

2. Interface déclarative

3. Orienté état (Level Based)

4. Contrôleurs Autonomes

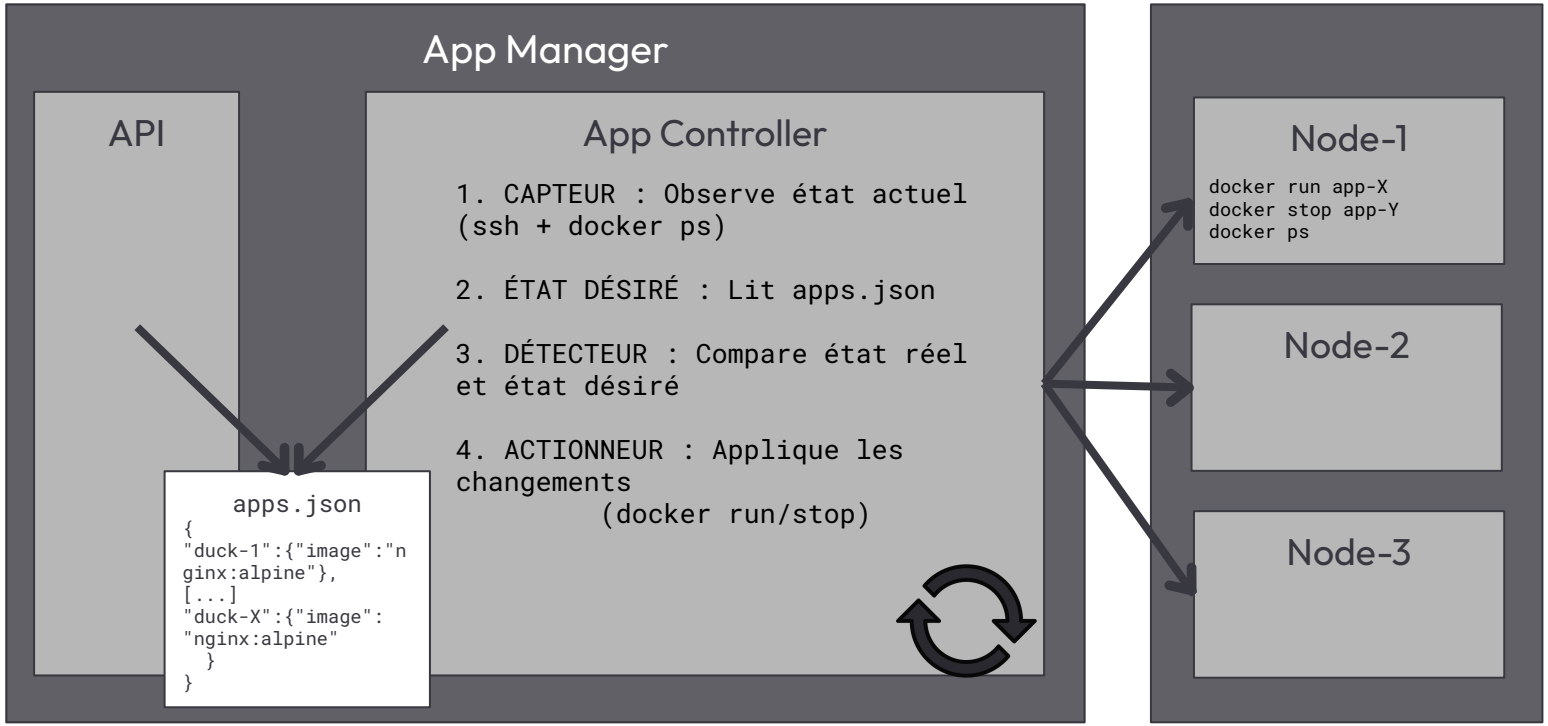
5. API centralisée et transparente

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

App Manager : version finale

Un orchestrateur qui applique nos 5 principes

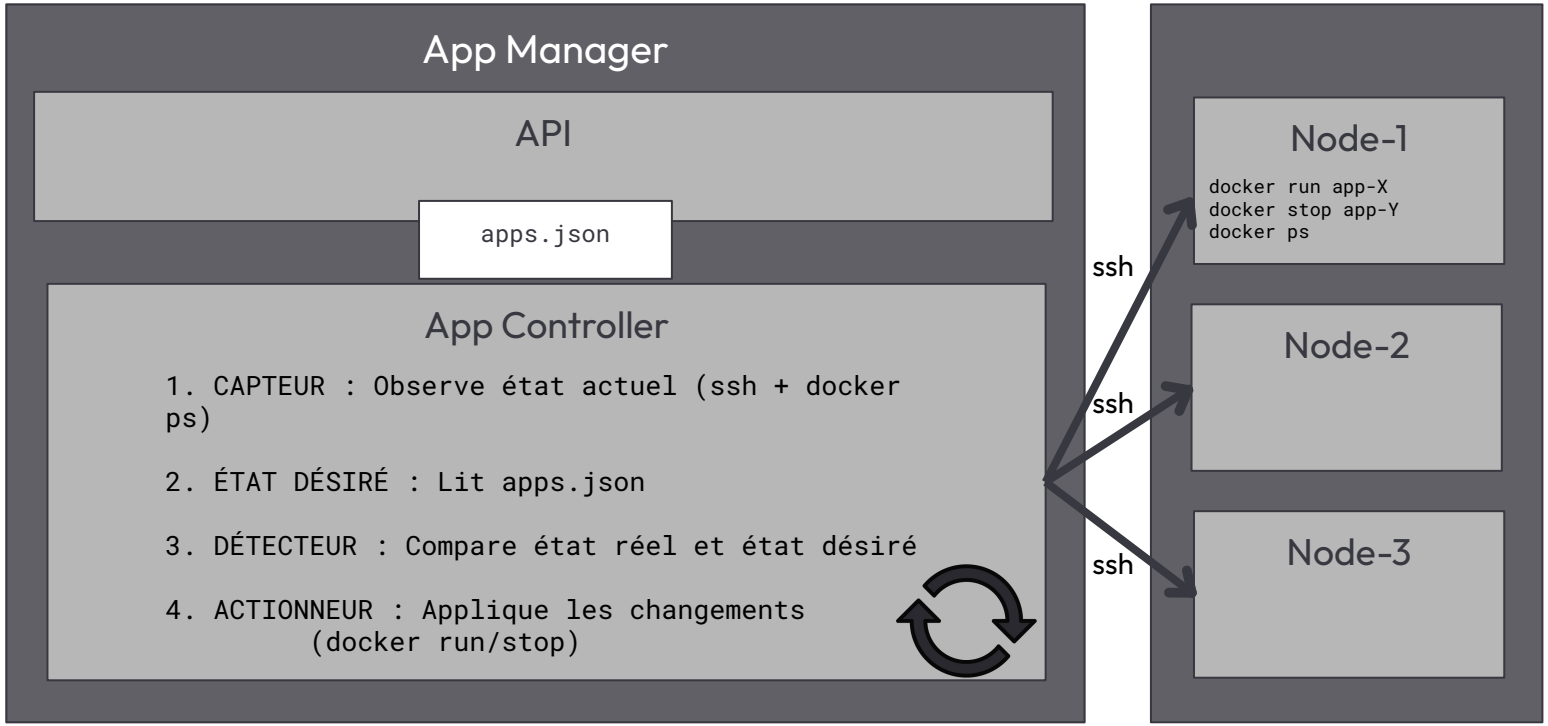
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

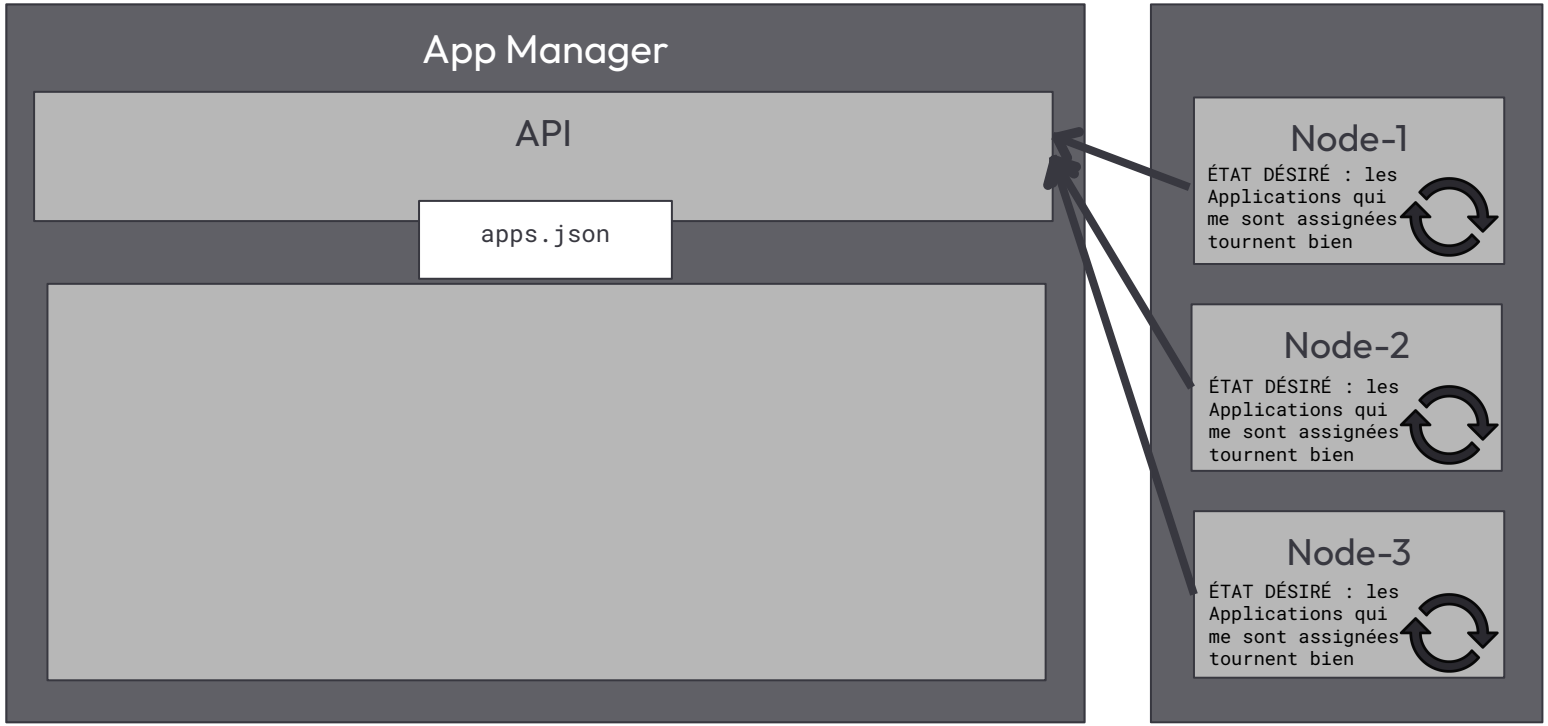
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

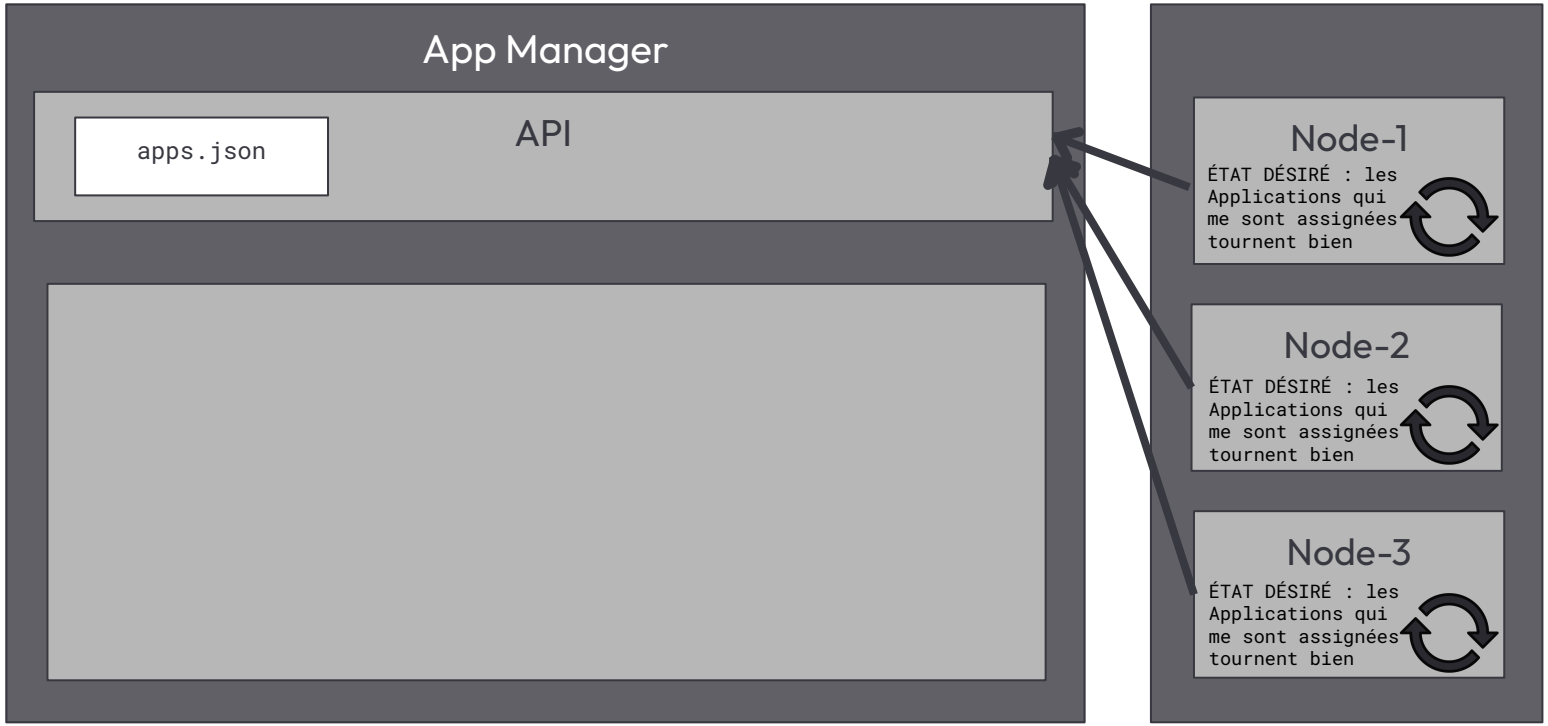
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

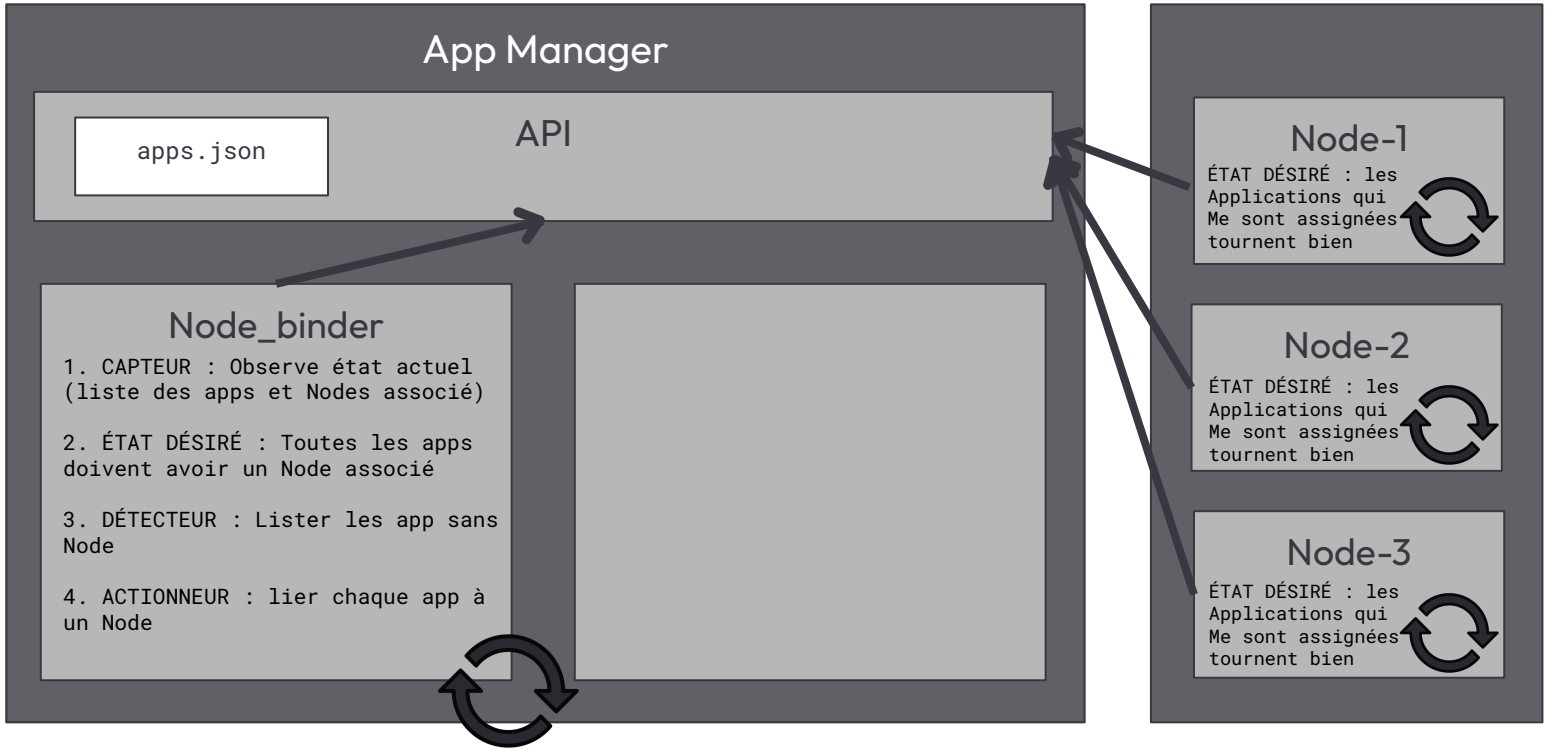
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

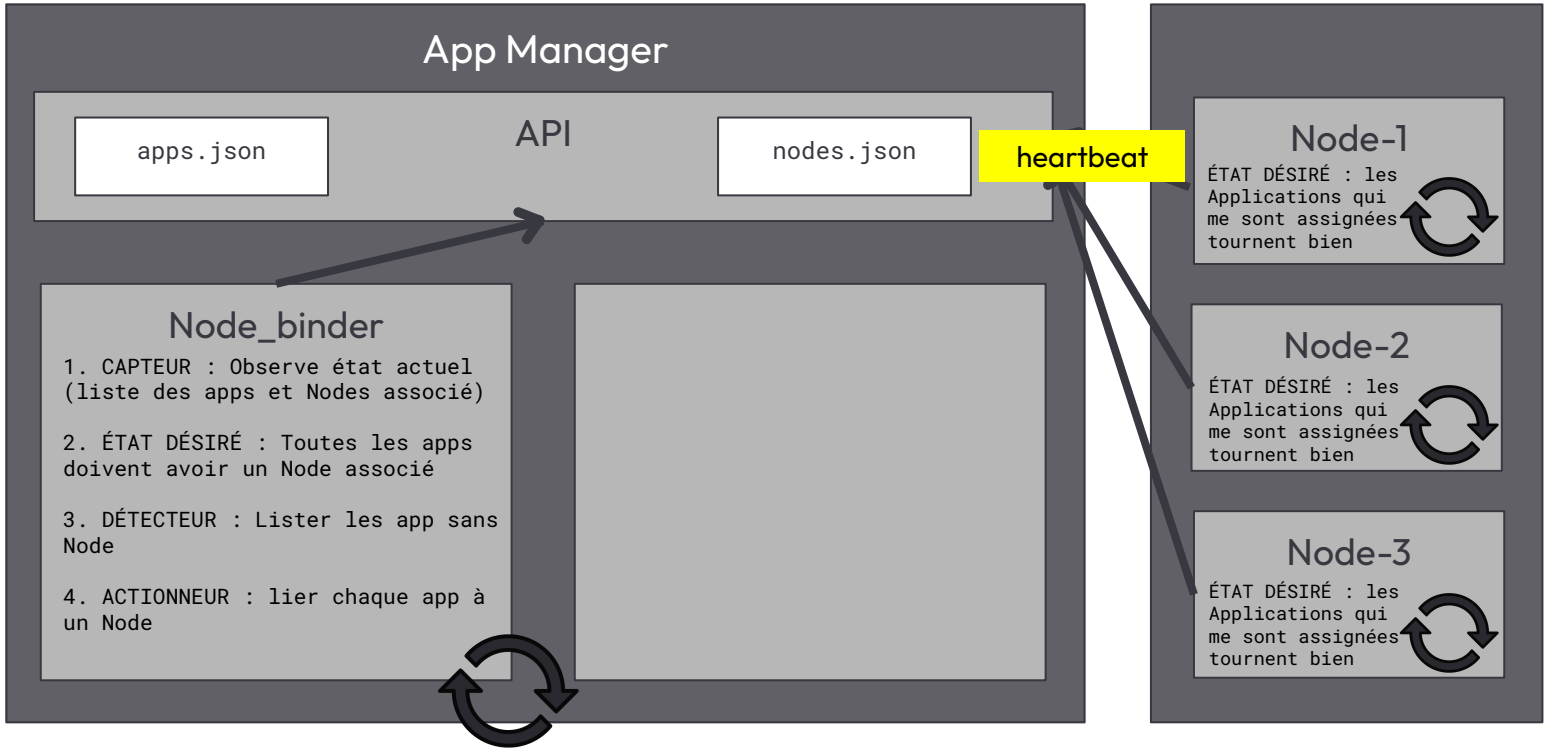
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

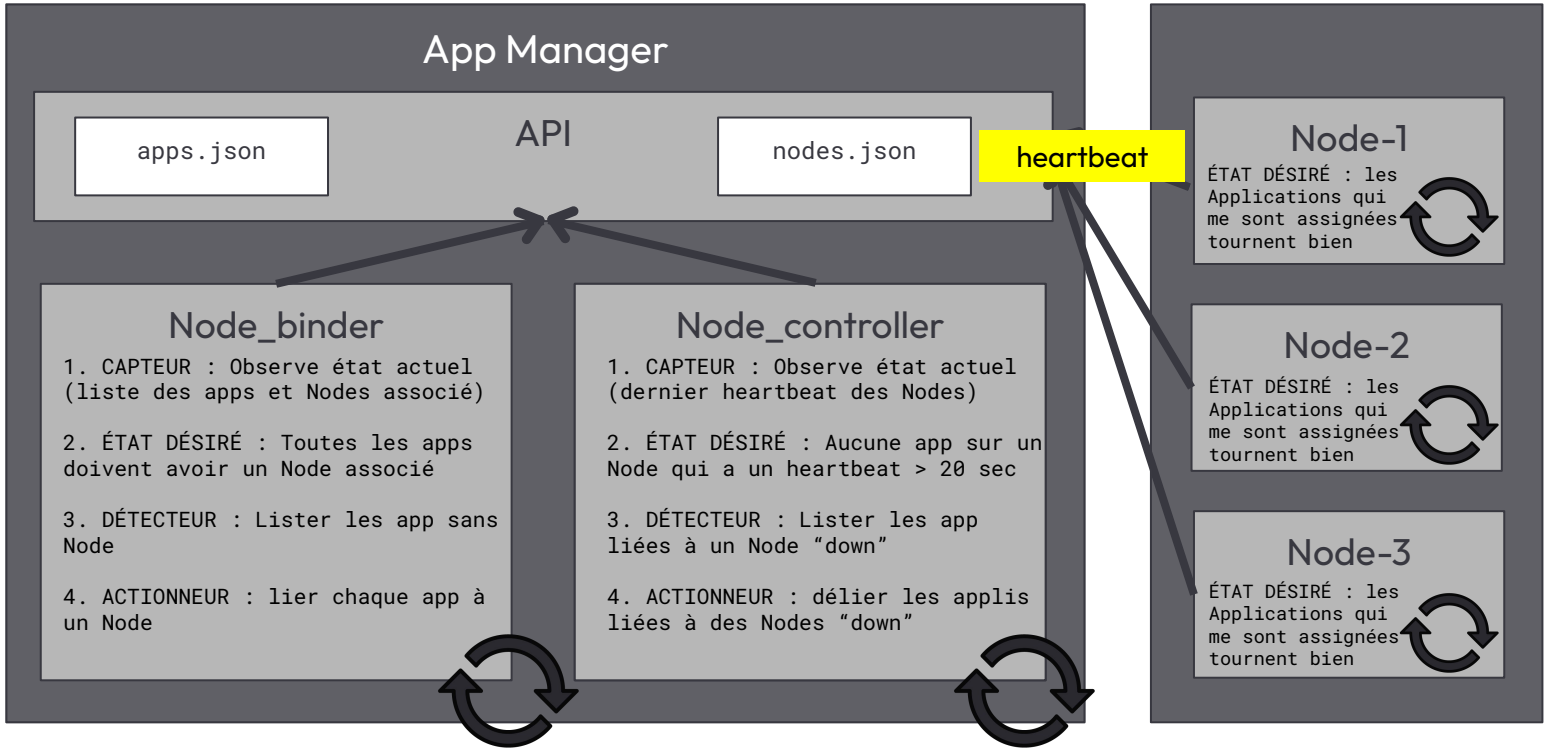
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

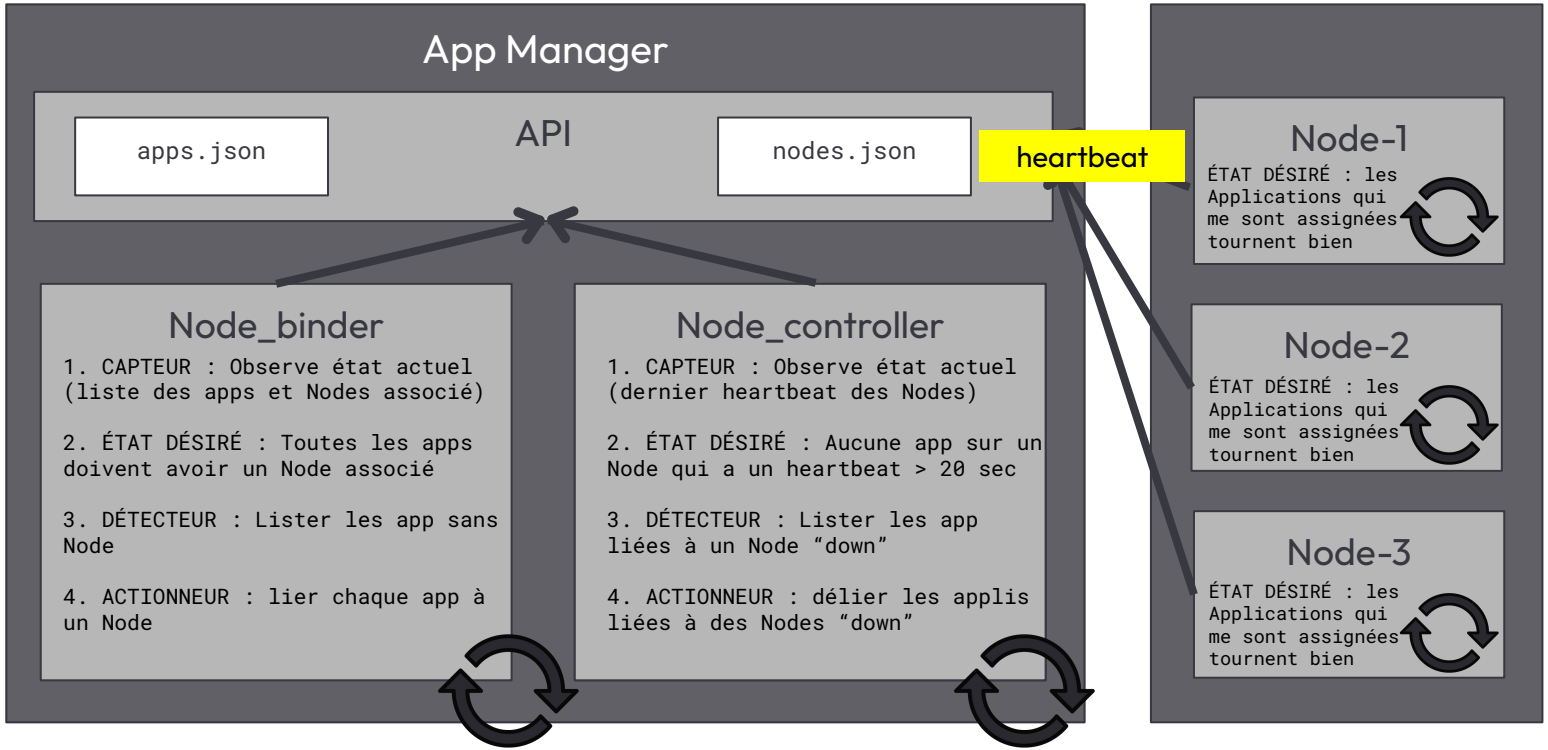
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

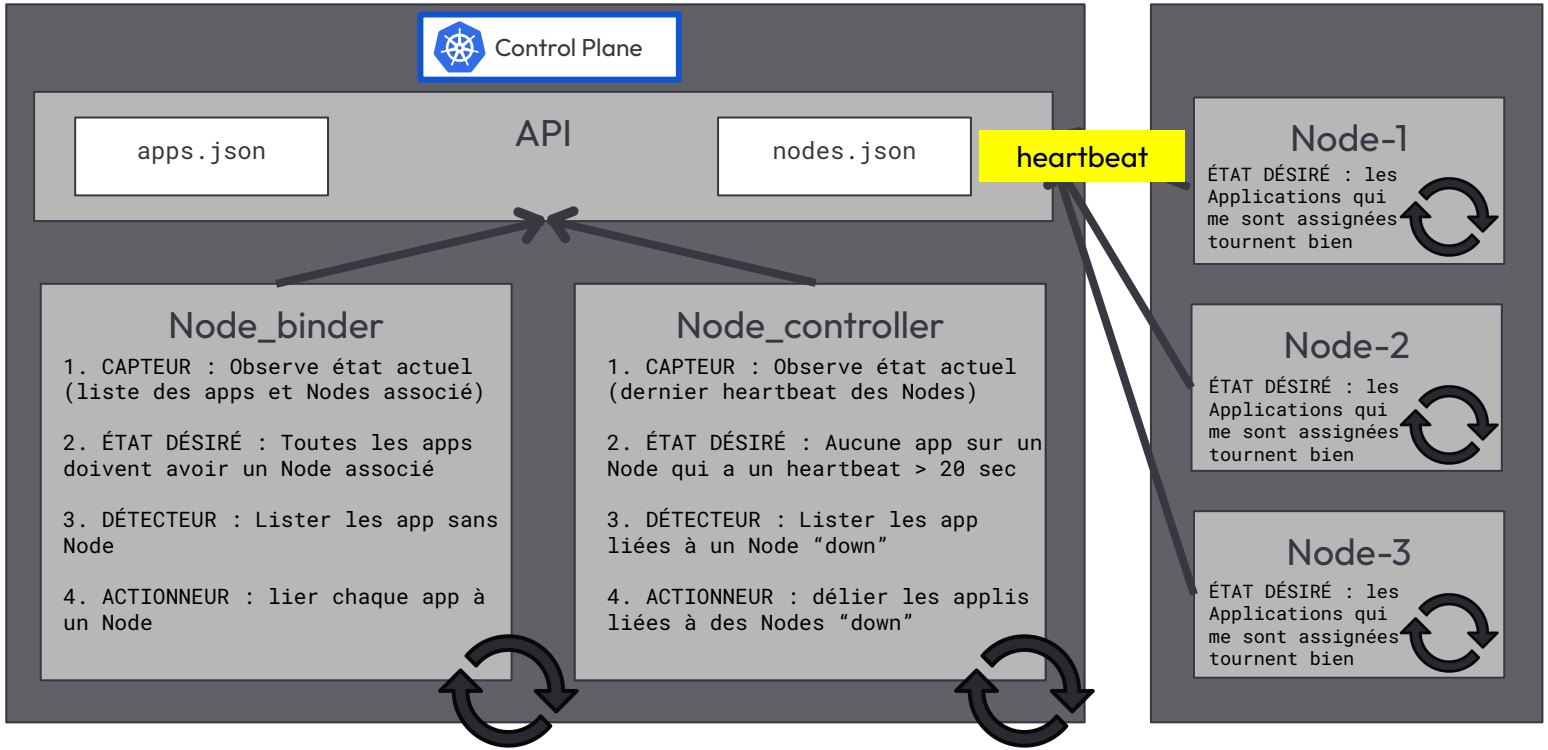
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

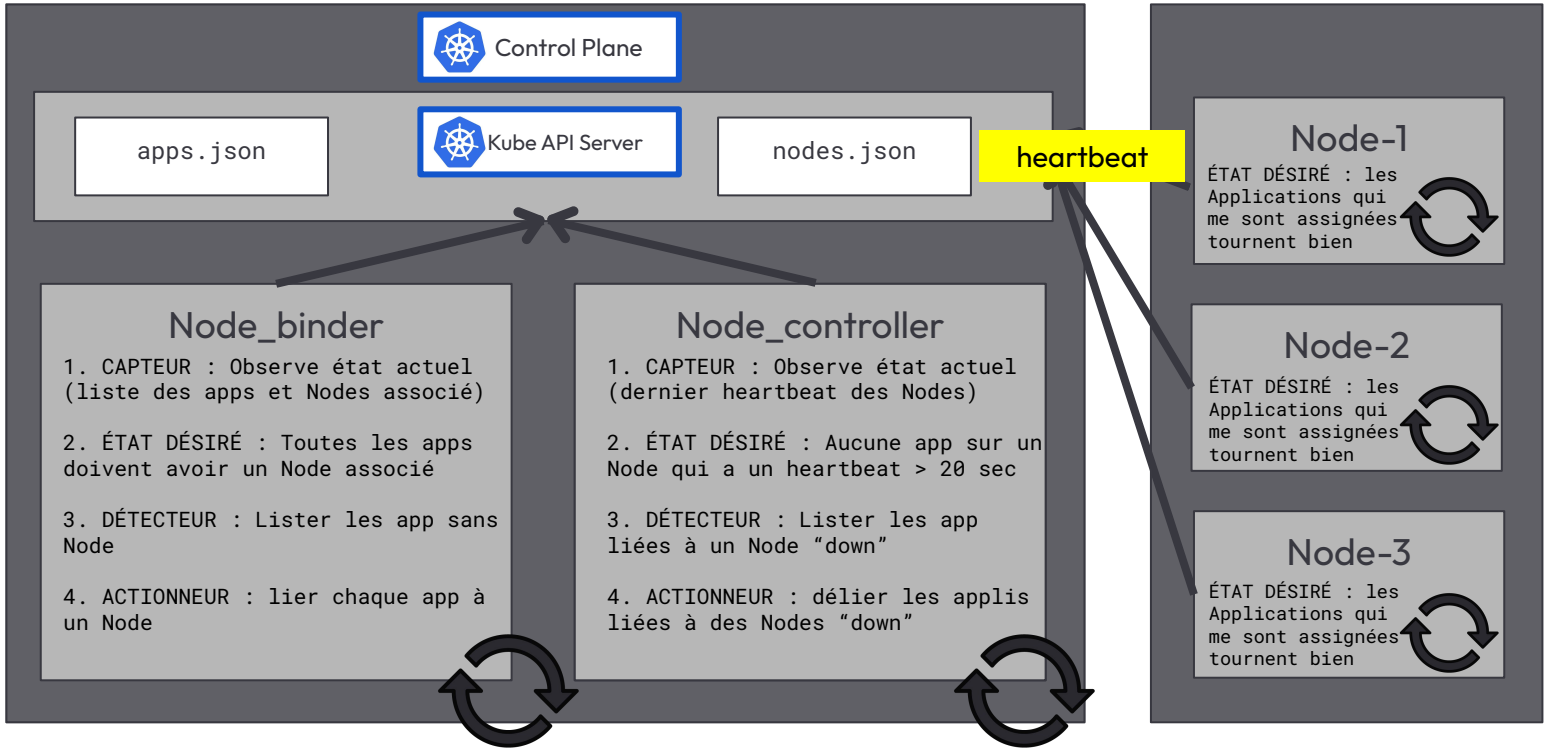
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

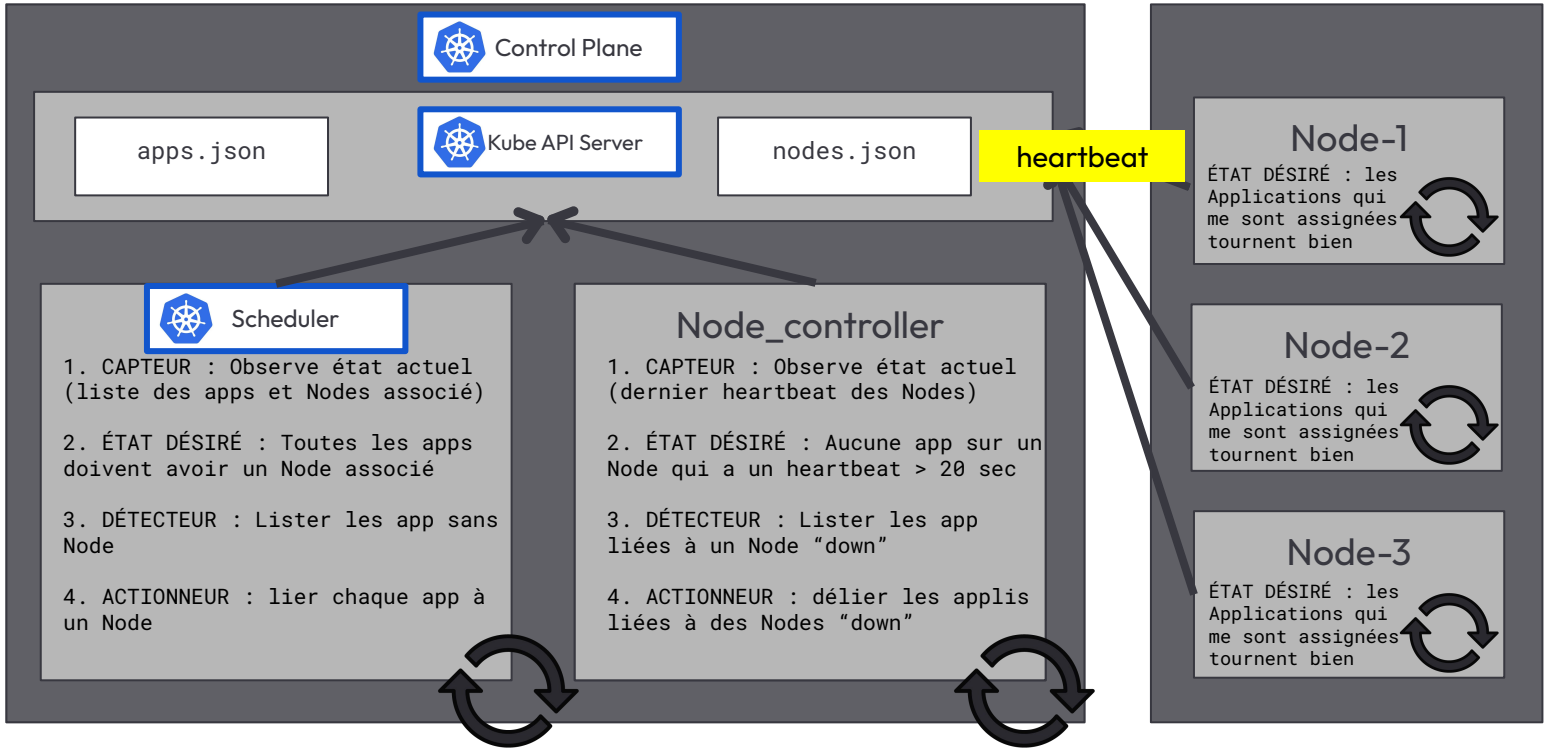
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

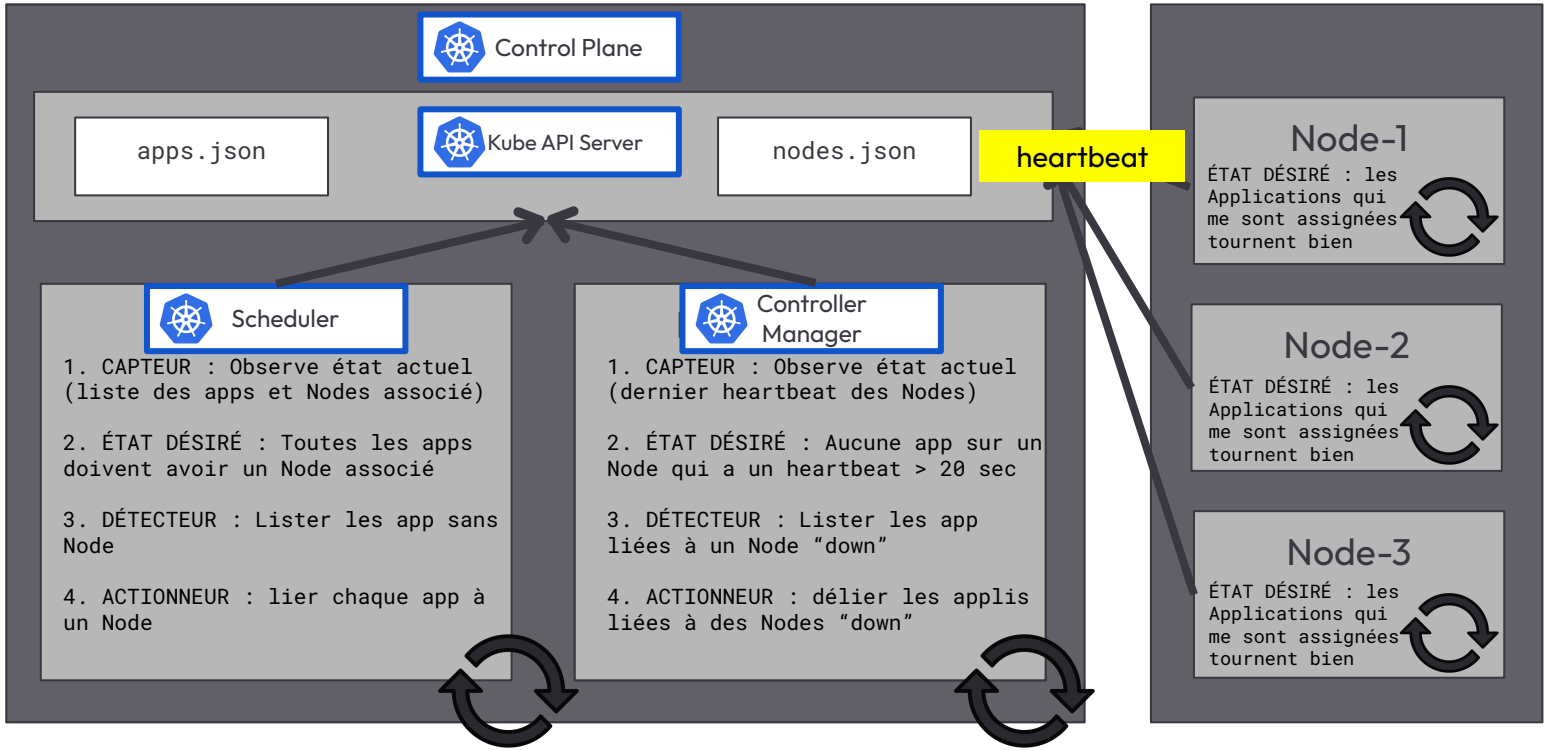
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

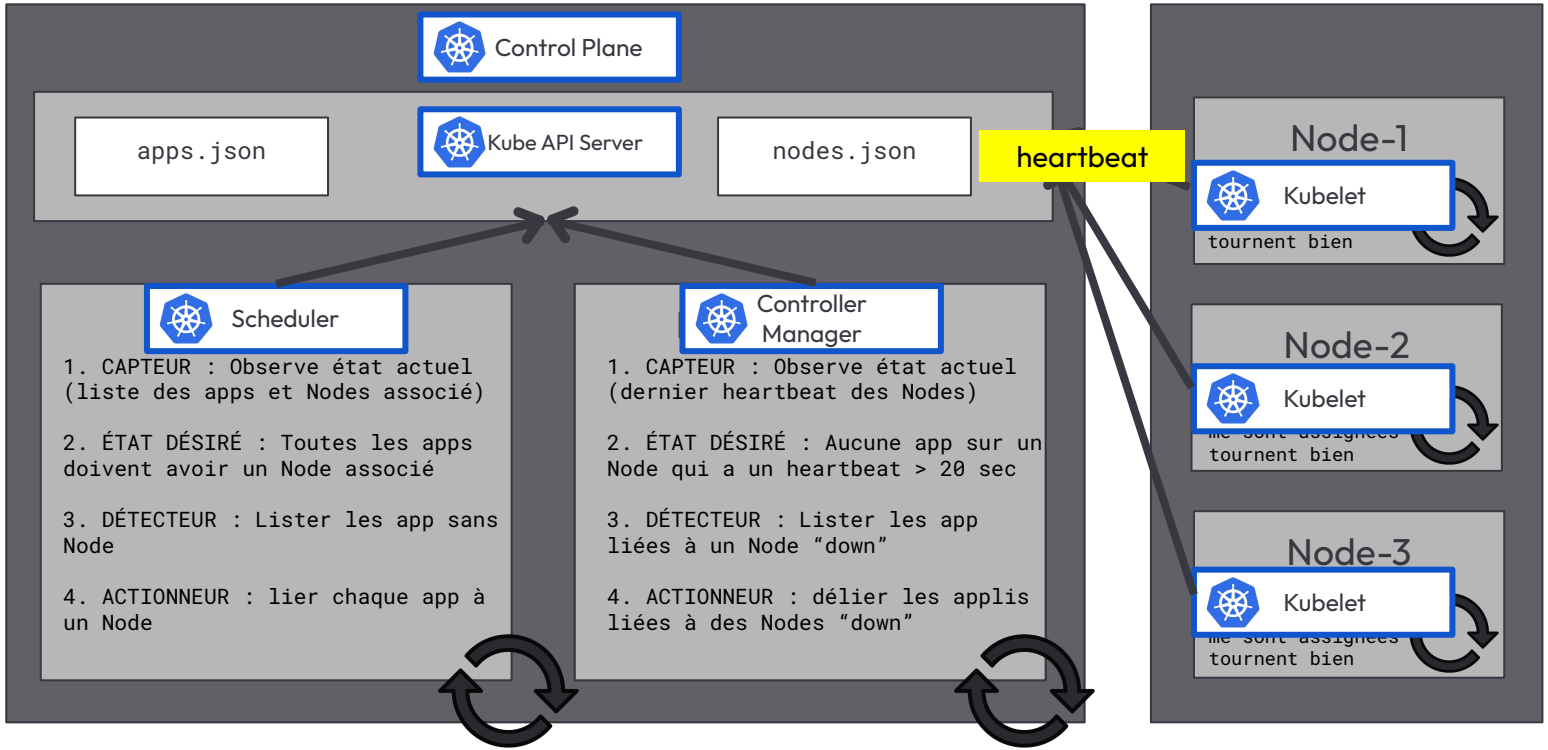
```
PUT /app/<name>
{,
"image": "nginx"
}
```



App Manager : version Finale

On applique nos 5 principes

```
PUT /app/<name>
{,
"image": "nginx"
}
```



1

Kubernetes n’est pas un outil de déploiement, **c’est un système de maintien en condition opérationnelle.**

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

2

Kubernetes n’est **pas un orchestrateur**

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

3

Kubernetes gère des **états désirés, peu importe l'objet**

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

Pour retrouver le code et les slides

Pour nous laisser un feedback

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

FIN.

Q/A

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved

Références

Kubernetes :

- Kubernetes - The documentary
- Joe Beda explains some of the inner workings of Kubernetes - Joe Beda
- The Technical History of Kubernetes - Brian Grant
- Borg, Omega, and Kubernetes - Google
- Understanding Kubernetes Through Real-World Phenomena and Analogies - Lucas Käldström
- Two reasons Kubernetes is so complex - Nelson Elhage
- Fundamentals of Declarative Application Management in Kubernetes - Zhang Lei
- Kubernetes Design Principles - Kubernetes Github repository
- Building Kubernetes (a lite version) from scratch in Go - Owumi Festus
- The Kubernetes Resource Model (KRM) - Kubernetes Github repository

Level Trigger vs. Edge Trigger :

- Level Triggering and Reconciliation in Kubernetes - @jbows

Control Theory :

- Applying control theory concepts in software applications - Dr. Wolfgang Winter
- Reconciliation Pattern, Control Theory and Cluster API: The Holy Trinity - Sachin Singhy
- Desired State How React, Kubernetes and control Theory have lots in common - Branislav Jenco
- Control Theory in Container Fleet Management - Vallery Lancey

Promise Theory :

- Thinking in Promises - Mark Burgess

LA DUCK CONF BY OCTO TECHNOLOGY © 2026 - All rights reserved